

From Autonomous Quantitative Research to QuantConnect / LEAN

A Governed Semantic-Compilation Architecture for Multi-Model Equity Strategies

Research Objective → Governed Specification → Deterministic Compiler
→ LEAN Model Tournament → Portfolio Construction
→ Independent Risk → Orders / Fills → Audit / Human Review

Alejandro Reynoso

Technical Academic Paper and Implementation Companion

September 2026

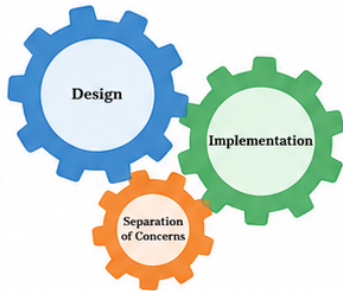
Research status. This paper documents the architecture encoded in the supplied NB07 governed QuantConnect / LEAN implementation bridge. The notebook marks the generated experiment as `READY_FOR_QUANTCONNECT_BACKTEST`; QuantConnect cloud compilation and platform backtest fields remain `NOT_RUN`. Accordingly, the document explains the algorithm, design, governance, and implementation, but does not infer platform performance that has not yet been observed.

Visual Overview: From Concept to Code

From Concept to Code

A Structured Approach to Software Development

Separation of Concerns · Thoughtful Design · Clean Implementation



A Practical Guide with Diagrams and Examples

Your Name

\today

From Concept to Code

1. The Process Overview

1 The Development Process

We follow a clear, iterative process that takes a problem from initial understanding to a working, maintainable solution. The process emphasizes separation of concerns, thoughtful design, and a clean implementation.

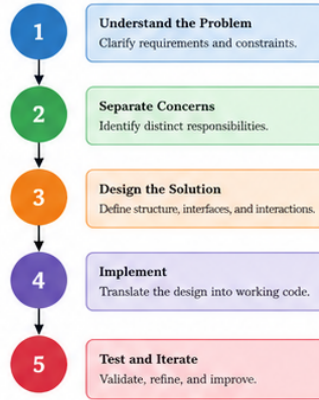


Figure 1: Overview of the development process.

i Key Idea

A well-structured process leads to software that is easier to understand, maintain, and extend.

1

From Concept to Code

2. Separation of Concerns

2 Separation of Concerns

Separation of concerns (SoC) is a design principle that divides a system into distinct parts, each with a single responsibility. This reduces complexity, improves maintainability, and enables independent development and testing.

Without Separation of Concerns	With Separation of Concerns
✗ Mixed responsibilities	✓ Clear, focused responsibilities
✗ Tight coupling	✓ Looser coupling
✗ Harder to test and maintain	✓ Easier to test and maintain
✗ Changes in one part affect others	✓ Changes are localized
✗ Lower reusability	✓ Higher reusability

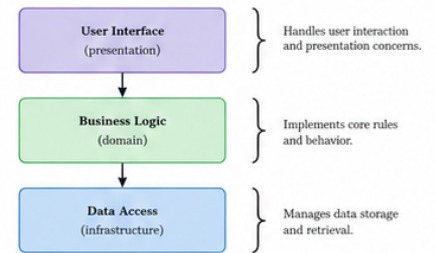


Figure 2: Example separation of concerns in a layered architecture.

2

From Concept to Code

3. Design

3 Design

Good design provides a blueprint for implementation. It defines the structure of the system, the key components, their responsibilities, and how they interact.

3.1 Design Goals

- 1 **Modularity** – Components with clear responsibilities.
- 2 **Extensibility** – Easy to add new features.
- 3 **Maintainability** – Simple to understand and modify.
- 4 **Testability** – Support for isolated testing.

3.2 Example Design: Task Manager

We will design a simple task management application to illustrate the concepts.

3.2.1 Component Diagram

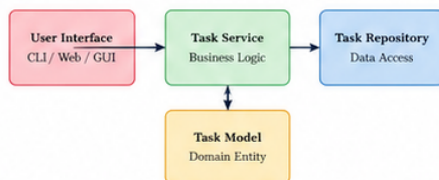


Figure 3: High-level design of the task manager.

3

From Concept to Code

4. Implementation

4 Implementation

With a clear design in place, we can implement the solution. The code below shows a simple implementation of the task manager using Python, organized according to our separation of concerns.

4.1 Project Structure

```
task_manager/  
├── ui/  
│   └── cli.py           # User interface  
├── service/  
│   └── task_service.py  # Business logic  
├── repository/  
│   └── task_repository.py # Data access  
└── model/  
    └── task.py          # Domain model
```

Figure 4: Project structure.

4.2 Key Components

Domain Model (model/task.py)

```
python  
  
from dataclasses import dataclass  
  
@dataclass  
class Task:  
    id: int  
    title: str  
    completed: bool = False
```

Figure 5: Task domain model.

4

From Concept to Code

5. Summary

5 Summary

By following a structured process, separating concerns, designing thoughtfully, and implementing with clarity, we can build software that is robust, maintainable, and easy to evolve.

✓ Key Takeaways

- 1 Follow a clear, iterative development process.
- 2 Separate concerns to reduce complexity.
- 3 Invest in a good design.
- 4 Implement with the design in mind.
- 5 Test, iterate, and improve.

Next Steps

- Apply these principles to a more complex project.
- Explore design patterns and architectural styles.
- Automate testing and integrate with a CI/CD pipeline.
- Continuously refactor and improve.

“Good software is not accidental. It is the result of a deliberate process, a clear design, and a commitment to separation of concerns.”

5

Contents

1 Purpose and Audience	2
2 From the Notebook Sequence to an Executable Research Strategy	3
3 The Architectural Principle: Semantic Compilation	5
4 Infographic: From Research Design to LEAN Execution	6
5 Separation of Concerns as a Financial-Systems Design Principle	9
6 Design Layer: The Strategy Contract	12
6.1 Mission and hard gate	12
6.2 Approved model, feature, and portfolio registries	12
6.3 The compiled specification	12
7 Design-to-Implementation Mapping	14
8 Implementation Layer I: Universe and Data Semantics	17
9 Implementation Layer II: Point-in-Time Feature Engineering	18
10 Implementation Layer III: The Five-Model Laboratory	20
11 Implementation Layer IV: Monthly Training and Deterministic Champion Selection	21
12 Implementation Layer V: From Champion Scores to a Cross-Sectional Signal	23
13 Implementation Layer VI: Portfolio Construction	24
14 Implementation Layer VII: Independent Risk as a Veto Layer	26
15 Implementation Layer VIII: Corporate Actions and Eligibility	28
16 Implementation Layer IX: Fees, Slippage, and Execution Reality	29
17 Runtime Scheduling and Event-Driven Execution	30
18 Falsification Before Performance Claims	31
19 Audit, Provenance, and Object Store Evidence	32
20 Semantic Reconciliation: Why the Notebook and LEAN Need Not Produce Identical Curves	33
21 What the QuantConnect User Is Actually Running	34
22 Why the Best Historical Model Is Not Hard-Coded	35
23 Where Artificial Intelligence Enters—and Where It Does Not	36
24 Technical Limitations and Research Extensions	37
24.1 Universe construction	37
24.2 Chronological split by row count	37

24.3	Repeated model selection	37
24.4	Champion metric	37
24.5	Forecast and holding horizons	37
24.6	Costs and shorting	37
24.7	Risk monitoring frequency	37
24.8	Factor neutrality	37
24.9	Audit hardening	38
25	Practical QuantConnect Installation and Research Workflow	39
26	A More Institutional Roadmap	40
27	Conclusion	41
A	Algorithmic Summary	44
B	Compiled QuantConnect Configuration	46
C	Generated LEAN Algorithm	48

List of Figures

1	End-to-end governed process from research objective to LEAN execution, audit evidence, and human review.	7
2	Separation of concerns. Each layer owns a bounded responsibility and cannot silently absorb the authority of another layer.	10
3	Design-to-implementation mapping. Every major research contract has a LEAN representation and a verification mechanism.	15

Abstract

This paper presents a stand-alone technical explanation of a governed machine-learning equity strategy implemented for QuantConnect / LEAN. The implementation is the operational endpoint of a broader quantitative-research sequence that began with controlled data and progressed through point-in-time feature engineering, heterogeneous predictive models, portfolio construction, execution realism, independent risk, reusable tools, orchestration, closed-loop research, and bounded large-language-model participation. The intended reader is a QuantConnect practitioner who may not wish to reproduce the full notebook sequence but does want to understand exactly what algorithm is being implemented, why its architecture looks different from a conventional “model-to-trade” script, and how research semantics are preserved when they are translated into an event-driven engine.

The central architectural concept is *semantic compilation*. A human research mission, and optionally an LLM-assisted research proposal, is not allowed to become execution authority. Instead, the research control plane produces a governed `StrategySpec` containing the universe, data semantics, feature contract, model families, training schedule, champion-selection rule, signal transformation, portfolio method, cost assumptions, risk limits, corporate-action policy, audit requirements, and explicit authority denials. A deterministic compiler validates the proposal against approved allowlists and emits a LEAN project composed principally of `main.py` and `strategy_config.py`.

The generated algorithm uses a static teaching universe of thirty U.S. equities divided into six sector groups. Nine trailing market features are constructed from adjusted daily prices and volumes. Five model families—logistic regression, k-nearest neighbors, random forest, multilayer perceptron, and linear regression—are retrained monthly on a pooled panel. A chronological validation segment determines the current champion using balanced classification accuracy and a fixed deterministic tie-break order. The champion produces cross-sectional scores, approximately the highest 20% of eligible securities are selected for the long side and the lowest 20% for the short side, and inverse-volatility weighting generates preliminary exposures. An independent logical risk governor then enforces per-position, sector-gross, total-gross, and portfolio-drawdown controls before approved weights reach `set_holdings`. Dividends and splits are processed as platform events, a two-calendar-day cool-down affects security eligibility, and fee/slippage reality models are explicitly attached to each security.

The paper emphasizes separation of concerns. The LLM/control plane can assist with semantic interpretation and hypothesis expansion; the deterministic compiler controls what may be executed; the model laboratory controls predictive competition; the portfolio layer converts scores into exposure; independent risk can clamp or veto the portfolio; LEAN owns market time, events, orders, fills, and accounting; and human authority remains necessary for promotion. The result is not simply a machine-learning strategy, but a governed model-selection and implementation process designed to remain inspectable, falsifiable, and auditable.

The deployed research object is not a single preferred model. It is a governed sequence:

objective → validated specification → model competition
→ portfolio proposal → risk review → LEAN execution.

AI can enlarge and interpret the research space, but deterministic rules, risk controls, platform mechanics, audit evidence, and human authority determine what the system is actually permitted to do.

1 Purpose and Audience

A QuantConnect user opening the generated project can understand the trading logic without reproducing the entire research curriculum. At the most compact level, the strategy is a monthly retrained, cross-sectional, long/short U.S. equity algorithm. Thirty securities are represented through trailing momentum, volatility, volume, and moving-average features. Five statistical and machine-learning models compete. The model that performs best on a later chronological validation segment becomes the current champion. At the monthly portfolio reconstruction, the champion scores eligible securities. The strongest scores are assigned to the long side, the weakest to the short side, inverse volatility determines preliminary position sizes, independent risk controls modify or reject those proposed weights, and only the approved result reaches the LEAN order interface.

That description is operationally correct, but it is not yet the design philosophy. The distinguishing feature of the implementation is that the algorithm did not begin with a declaration that Random Forest, KNN, a neural network, or any other model was “the strategy.” The surrounding notebook sequence progressively separated the research institution into layers: data, features, candidate models, signals, portfolio construction, execution assumptions, risk, governance, tools, agents, mission control, falsification, audit, and bounded LLM reasoning. NB07 is the bridge that translates those research semantics into a platform-executable form.

The paper therefore has two objectives. First, it reconstructs the algorithm for a reader who wants a complete technical understanding of the implemented strategy. Second, it explains why the implementation contains elements that a conventional teaching backtest might omit: a machine-readable strategy contract, allowlists, hashes, a mission gate, an explicit risk governor, corporate-action eligibility rules, runtime audit records, an Object Store bundle, and repeated denial of live-trading authority.

The intended audience also includes research managers, model validators, risk professionals, and software engineers who must decide whether a promising notebook can become a controlled trading experiment. Each group asks a different question. The researcher asks whether the predictive hypothesis is represented faithfully. The validator asks whether future information can leak into training or selection. The risk reviewer asks whether a forecast can create an exposure that exceeds policy. The engineer asks whether state transitions are deterministic and observable. The manager asks whether a result can be reproduced and attributed to a specific configuration. The architecture is designed so that these questions can be answered from explicit artifacts rather than from the memory of the original author.

The discussion deliberately distinguishes a research backtest from a production trading system. A successful cloud backtest would establish that the generated project compiles, consumes platform data, emits orders, and produces measurable behavior under declared assumptions. It would not establish economic durability, capacity, operational readiness, or permission to trade live capital. Those conclusions require additional out-of-sample evidence, sensitivity analysis, cost and borrow studies, operational testing, and formal approval. Sophisticated code and attractive statistics can create a false impression of completion; here, every result is evidence for a later decision rather than an automatic authorization.

Readers can therefore use the paper both as an algorithm map and as a review checklist. They can compare the declared universe, features, labels, models, schedules, portfolio rules, risk limits, execution assumptions, and audit outputs with the generated source. Any unexplained mismatch is a defect to investigate. This remains useful even when individual numerical choices are replaced, because the central concern is the traceable relationship among research intent, executable behavior, and retained evidence.

2 From the Notebook Sequence to an Executable Research Strategy

The notebook progression can be understood as a sequence of institutional questions. The early stages ask whether the system possesses a controlled market substrate and point-in-time information. Later stages ask which predictive representations should be compared, how forecasts become investable signals, how capital should be allocated, how costs and market mechanics affect realized results, and whether an independent risk layer can veto a strategy even when the predictive model remains confident. Subsequent stages convert analytical capabilities into reusable tools, organize those tools into research workflows, introduce mission state, provenance, and closed-loop orchestration, and eventually allow an LLM to interpret an ordinary-language research objective without giving that language model order authority.

NB07 occupies a different position. It asks what happens when the institution must leave the notebook environment and express itself inside an event-driven trading engine. A notebook usually reasons over completed matrices. LEAN experiences time sequentially. Historical data are requested from the current simulation state; scheduled callbacks determine when training and rebalancing occur; dividends and splits arrive as events; orders have tickets and fill events; portfolio value is updated by executed positions; and costs arise from reality models. The transition is therefore not a line-by-line code translation. It is a change of computational semantics.

The bridge preserves the research doctrine rather than the exact representation. The controlled database becomes a platform-native history contract. A vectorized feature table becomes a trailing feature-engineering routine. A notebook model tournament becomes a scheduled train callback. A portfolio vector becomes target holdings submitted through LEAN. Corporate-action records become event handlers. A local backtest cost subtraction becomes explicit fee and slippage models. Provenance becomes a combination of stable configuration hashes, experiment metadata, runtime event records, and an Object Store audit artifact.

Table 1 compresses the conceptual lineage for readers who enter directly at the QuantConnect layer.

Table 1: Research progression and what survives in the LEAN implementation.

Research layer	Institution-level question	LEAN expression
Controlled data	Can the evidence and its timing be trusted?	Declared normalization, trailing history, event-time semantics.
Competing models	Which representation generalizes best?	Five approved model families retrained in a monthly tournament.
Portfolio construction	How do forecasts become capital?	Cross-sectional ranking plus inverse-volatility weighting.
Independent risk	Can the strategy be vetoed?	<code>IndependentRiskGovernor</code> .
Tools / orchestration	Can approved capabilities be selected rather than improvised?	Model, feature, and portfolio allowlists compiled into configuration.
Closed-loop research	Can missions have state, provenance, and falsification?	Mission gate, deterministic compiler, validation report, experiment registry.
LLM research planning	Can natural language expand research without controlling trades?	Optional notebook-side planner; no LLM code in LEAN runtime.
Audit / governance	Can execution evidence be reconstructed?	Runtime records, hashes, Object Store bundle, human promotion authority.

This progression explains why apparently administrative artifacts are first-class research outputs. A feature name, calendar convention, validation split, or risk threshold may look like configuration detail, yet changing any one of them can alter the hypothesis being tested. The notebook sequence converts tacit choices into explicit contracts so that the final bridge does not have to guess what the author intended. When a choice remains unresolved, the correct response is to expose the ambiguity or reject the build rather than silently invent a convenient default.

The bridge belongs to a feedback loop. Platform observations reveal whether the translation behaves as expected;

reconciliation identifies semantic or mechanical differences; falsification tests challenge assumptions; and approved revisions return through the specification and compiler. The system may propose another experiment after a failure, but only a newly validated specification may alter what the next generated project is allowed to do. This preserves the distinction between learning from evidence and changing execution authority.

3 The Architectural Principle: Semantic Compilation

The central pedagogical idea of the bridge is *semantic compilation*. The notebook does not treat an LLM response, a research idea, or even a structured JSON object as executable merely because it is machine-readable. Instead, the research control plane creates an intermediate representation—the governed `StrategySpec`—that states what the experiment means. Only a deterministic compiler may translate that representation into platform code.

The conceptual pipeline is

Human Objective → Research Proposal → Validated `StrategySpec`
→ Deterministic Compiler → LEAN Project.

The strategy specification includes the research question, asset class, universe, universe groups, universe mode, resolution, calendar, timezone, data normalization, feature contract, label horizon, candidate model families, champion rule, training lookback, training schedule, signal rule, position rule, portfolio method, rebalance schedule, cost model, slippage model, risk limits, corporate-action policy, sample dates, starting cash, benchmark, warm-up period, random seed, expected outputs, known limitations, and authority settings. The design therefore treats these assumptions as first-class configuration rather than as scattered source-code details.

This is the same governance principle that makes bounded LLM use possible. An optional LLM can help interpret a natural-language research objective, but the output remains a proposal. The plan is checked against approved model, feature, and portfolio registries. Promotion must remain `HUMAN_REVIEW_REQUIRED`. Live trading and deployment must remain false. Unknown tools are rejected. The compiler then chooses an allowed portfolio method from the proposal and emits a complete specification.

A compiler does not ask whether a proposal sounds persuasive. It asks whether the proposal is syntactically and semantically valid under a declared language. NB07 applies that idea to quantitative research: the `StrategySpec` is the intermediate representation, the allowlists define the permitted language, and the LEAN project is the target program.

Semantic compilation is valuable when two implementations are syntactically different yet economically equivalent, or syntactically similar yet economically different. A vectorized notebook may compute a feature for every row at once, whereas LEAN computes the same quantity from trailing history at a scheduled event. The programs need not share control flow. They must share the information set, lookback convention, adjustment policy, missing-data rule, and decision time. Conversely, copying a familiar indicator expression into both environments proves little if one includes the current close while the other acts before that close is available. The compiler preserves meanings and timing constraints rather than performing a literal transcription.

The compilation boundary also creates a review surface. Inputs are declarative approved identifiers, limits, schedules, and policies; outputs are executable modules, reports, and hashes. Reviewers can ask whether every declared field has an unambiguous implementation site. A field that is accepted but ignored creates false confidence. Material hard-coded behavior without a specification source is equally dangerous because it cannot be governed upstream. A rigorous compiler should reject unknown fields, validate ranges and cross-field consistency, use stable ordering, and report what it generated.

Determinism does not promise identical numerical backtests across every future data or platform version. Data revisions, library changes, and fill-model differences can matter. It promises that a fixed approved input, compiler version, and environment define a reproducible program without discretionary agent intervention. This narrower guarantee localizes disagreements, enables meaningful hashes, and prevents conversational suggestions from mutating execution logic between review and run. Semantic compilation is therefore both an engineering pattern and a governance control.

4 Infographic: From Research Design to LEAN Execution

Figure 1 presents the entire process as a single governed pipeline. The most important visual feature is the prohibited direct path between the planner and the order interface. The LLM/control plane can influence what is researched, but it cannot bypass validation, model competition, portfolio construction, or independent risk.

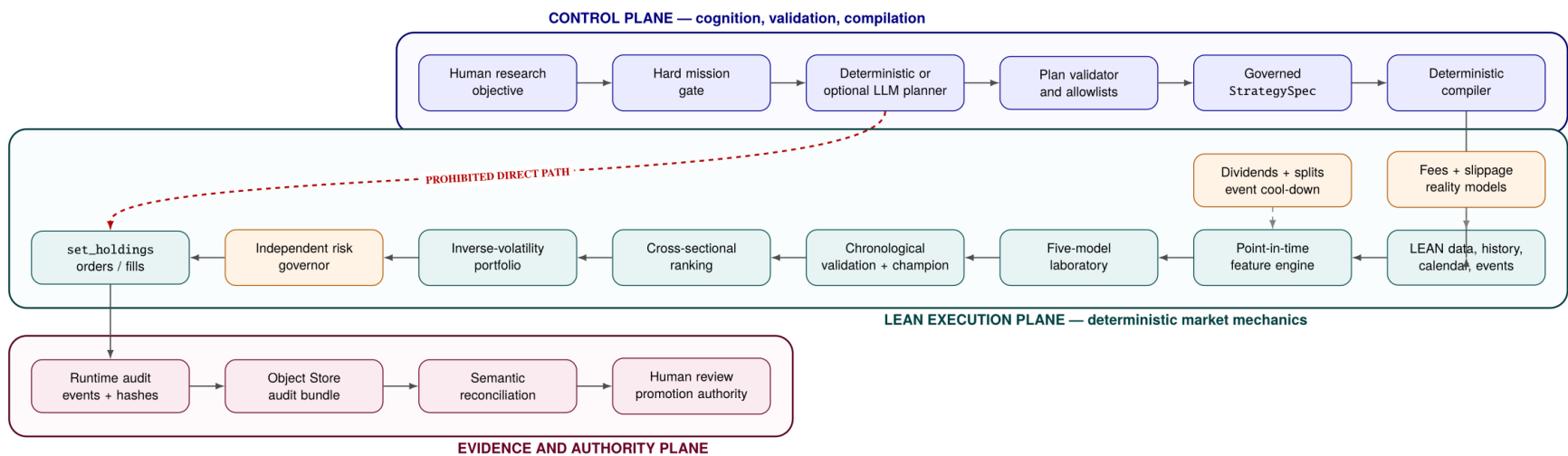


Figure 1: End-to-end governed process from research objective to LEAN execution, audit evidence, and human review.

The top plane is the cognitive and compilation environment. It accepts the research objective, applies a hard gate, optionally uses an LLM to propose a bounded plan, validates that plan, compiles the `StrategySpec`, and generates deterministic code. The middle plane is the LEAN runtime: market data, point-in-time feature engineering, the five-model laboratory, mathematical champion selection, signal generation, portfolio construction, independent risk review, and order execution. The bottom plane records evidence and preserves authority boundaries through audit events, stable hashes, the Object Store bundle, semantic reconciliation, and human promotion review.

The visual is not a simple pipeline in which every stage merely transforms data. Several stages are gates. Validation may reject a specification. The model tournament may fail to produce an eligible champion. Risk may reduce every proposed position to zero. Human review may decline promotion even when a backtest completes successfully. These veto points matter because a governed system is defined as much by the actions it can stop as by the analyses it can perform.

The upper band represents the research control plane, where objectives become bounded experimental choices. The center represents deterministic implementation, where those choices become scheduled calculations and explicit state. The lower band represents platform evidence, including events, orders, fills, holdings, diagnostics, and audit material. These layers operate on different clocks: research design changes between experiments, training occurs monthly, risk state changes with portfolio value, and the engine processes events sequentially. The diagram compresses those clocks into one view while later sections explain their interaction.

Return arrows from verification to design are deliberate. A compiler can be correct while the hypothesis is weak, and a model can appear predictive while the implementation is semantically wrong. Reconciliation classifies these failures. Unexpected turnover may originate in order mechanics, ranking instability in a small validation sample, and a universe discrepancy in configuration. Treating them all as generic performance problems would obscure the remedy.

The infographic remains a navigation aid rather than a substitute for contracts and source. Colors communicate responsibility, but only machine-readable specifications and executable checks determine behavior. A reviewer should be able to move from any visual block to the corresponding section, configuration field, algorithm method, and audit record. That traceability is the criterion by which the overview should be judged.

One additional reading convention is useful: horizontal movement shows a proposed action progressing toward execution, whereas vertical separation shows that the action changes ownership. Crossing a boundary requires a defined artifact, such as a specification, score vector, proposed portfolio, risk decision, or fill record. This makes interfaces visible and gives reviewers concrete objects to compare instead of relying on informal assurances that adjacent components are aligned.

The figure also emphasizes that a safe stop is a valid terminal state. Rejection, abstention, or zero exposure can demonstrate correct governance even though no trade appears.

5 Separation of Concerns as a Financial-Systems Design Principle

The system deliberately assigns different responsibilities to components that have different comparative advantages. A language model is useful for semantic interpretation and hypothesis expansion. A deterministic validator is useful for enforcing permissions and invariants. Statistical and machine-learning estimators are useful for extracting regularities from historical data. A portfolio layer is useful for converting scores into economic exposure. A risk layer is useful for constraining that exposure independently of model enthusiasm. LEAN is useful for market time, event sequencing, orders, fills, and accounting. Human review is useful for fiduciary judgment and authority.

Figure 2 makes these boundaries explicit.

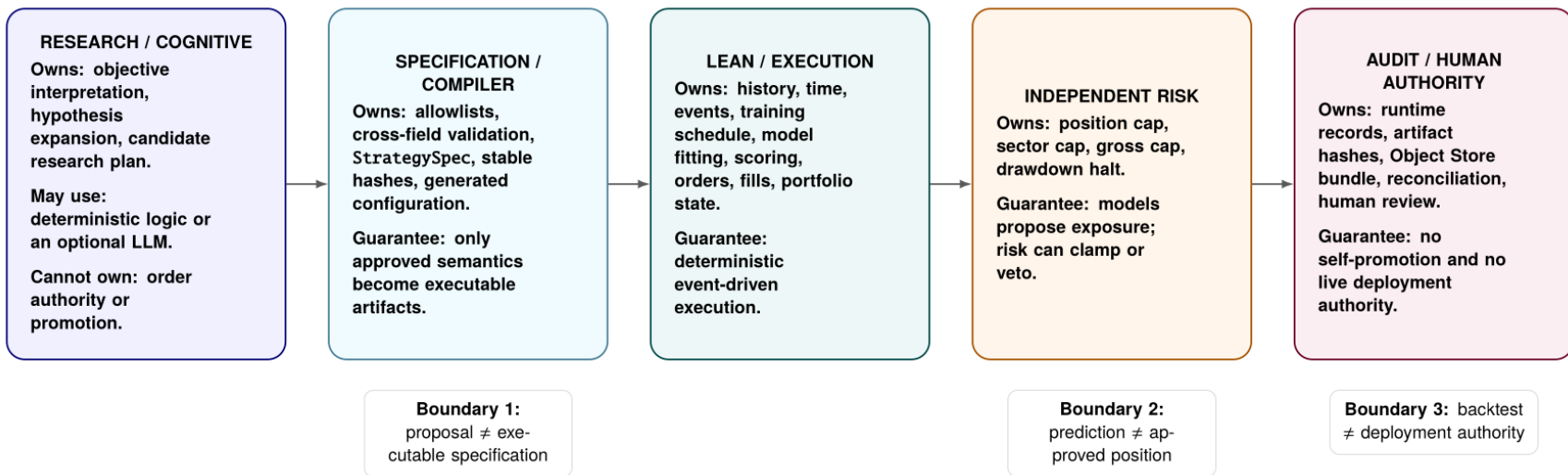


Figure 2: Separation of concerns. Each layer owns a bounded responsibility and cannot silently absorb the authority of another layer.

Three boundaries deserve special emphasis.

First, *proposal is not execution*. A planner can suggest approved models and experiments, but the proposal must pass deterministic validation and compilation.

Second, *prediction is not position*. A champion model produces scores, not orders. A cross-sectional signal rule and portfolio method translate the scores into preliminary weights; independent risk then has the right to clamp or eliminate them.

Third, *backtest evidence is not deployment authority*. The experiment registry may record that a platform backtest has been completed, but the notebook still requires human review for promotion and explicitly denies live execution.

This separation is not a stylistic software-engineering preference. It is a financial-governance architecture. In a production institution, the same boundaries can be made stronger by distributing them across services, accounts, permissions, broker controls, or teams. The teaching implementation keeps them inside one project where necessary, but preserves the conceptual ownership of each responsibility.

The boundaries limit the blast radius of a defect. A poor forecast may affect rankings but cannot rewrite risk limits. A portfolio-construction bug cannot alter historical labels. A fill-model difference should appear in execution evidence rather than silently changing the research hypothesis. An LLM-generated suggestion can enter the proposal space, but it cannot call the order interface or approve its own capabilities. Each component receives only the information and authority required for its role.

This resembles institutional investment practice, where independent functions evaluate the same action from different perspectives. Research explains why an exposure may be attractive, portfolio construction determines how it fits other exposures, risk tests concentration and loss constraints, and operations determines whether it can be executed and reconciled. Independence does not mean isolation; it means one function cannot silently absorb another's authority. The bridge preserves this principle within a compact implementation.

Separation improves testing. Feature calculations can be checked with fixed histories, model selection with synthetic scores and known ties, portfolio construction with predetermined scores and volatilities, and risk with portfolios that violate one limit at a time. Scheduling can be verified in event logs and execution through platform results. These tests are more diagnostic than a single end-to-end equity curve because each has a narrow expected outcome.

More interfaces do create artifacts to maintain. The appropriate response is to keep contracts small, typed, and purposeful. A boundary that adds no rule or evidence should be questioned. A boundary that prevents hidden coupling, records an authority decision, or permits isolated verification earns its complexity. The divisions used here correspond to materially different responsibilities and provide a defensible base for later expansion.

6 Design Layer: The Strategy Contract

6.1 Mission and hard gate

The mission encoded in the notebook asks whether a cross-sectional, event-aware U.S. equity strategy can produce robust risk-adjusted performance across sectors while comparing approved machine-learning families, using point-in-time trailing features, realistic cost assumptions, independent risk, corporate-action awareness, and falsification tests. The sample is declared as 2021-01-01 through 2025-12-31. The mission is research-only.

The hard gate requires nine mission fields and rejects prohibited external effects. The asset class must be U.S. equities; `research_only` must be true; `live_trading_authorized` and `deployment_authorized` must be false; and the promotion view must be `HUMAN_REVIEW_REQUIRED`. The mission therefore cannot gain execution authority by wording itself as a research task.

6.2 Approved model, feature, and portfolio registries

The approved model library is fixed to five families:

$$\mathcal{M} = \{\text{Logit}, \text{KNN}, \text{RF}, \text{MLP}, \text{LinearReg}\}.$$

The approved portfolio methods are equal weight, inverse volatility, a risk-parity proxy, and volatility targeting. The compiled plan chooses inverse volatility when it is present in the candidate set because the compiler applies a declared preference order. The approved feature contract contains nine variables:

$$\mathcal{F} = \{ret_1, mom_5, mom_{20}, vol_{20}, vol_{60}, Z_{20}^V, Z_{20}^{DV}, MA20Ratio, MA50Ratio\}.$$

The planner is not allowed to add an unregistered model, feature, or portfolio method. This is how the architecture converts an open-ended research interface into a bounded execution language.

6.3 The compiled specification

The compiled strategy is identified as `NB07_QC_GOVERNED_MULTI_MODEL_V2`, version `2.0.0-research`. It uses daily adjusted U.S. equity data, a 320-bar training lookback, monthly model retraining before the market open, monthly rebalancing after the market open, a one-day prediction horizon, and a warm-up of 80 daily bars. Starting cash is USD 1,000,000 and SPY is the benchmark.

Risk limits are part of the specification rather than being hidden in the risk class:

$$G_{max} = 1.00, \quad |w_i| \leq 0.10, \quad G_{sector} \leq 0.35, \quad DD_{halt} = 12\%.$$

The specification also declares an explicit constant fee model and a volume-share slippage model. The fixed fee parameter is currently USD 0.00 per order, while the slippage parameters are a 2.5% volume-share limit and a price-impact coefficient of 0.10. The zero fee must therefore be understood as a visible research calibration, not as proof that trading is costless.

Several contract fields must be interpreted jointly. A lookback is meaningful only with a bar frequency and adjustment policy. A training schedule is meaningful only with a decision timestamp and label horizon. A gross-exposure target is meaningful only beside position, sector, and drawdown limits. Validation therefore needs cross-field checks in addition to types and ranges. Required history must cover the longest feature window and forward label; the universe needs sector metadata when sector limits apply; and every model in the tie-break order must also appear in the approved model list.

The contract supplies a stable vocabulary for audit. Instead of logging that the system used “the usual momentum features,” runtime records can refer to exact feature identifiers and the configuration hash. Instead of describing risk as conservative, they can record proposed gross exposure, applied clamps, and final weights. This makes evidence comparable across runs and reduces dependence on undocumented conventions.

Versioning is essential because a specification is not timeless. Changing a feature formula, universe member, hyperparameter, or corporate-action rule creates a different experiment even if its name stays the same. The hash gives that experiment a compact identity, while preserved JSON supplies the human-readable detail. Both are necessary: a hash proves identity but cannot explain content.

Approval consequently attaches to a bounded specification and compiler behavior, not to a broad aspiration such as trading equities with machine learning. If a later cycle proposes another model or higher exposure, it must pass validation and review as a new object. This procedural discipline also protects scientific inference, because results can be attributed only when the tested object is stable enough to name, reproduce, and compare.

7 Design-to-Implementation Mapping

One of the risks in quantitative platform migration is that design concepts become distorted when expressed through platform APIs. Figure 3 shows how each declared design contract maps into a concrete LEAN implementation and then into a falsification or verification test.

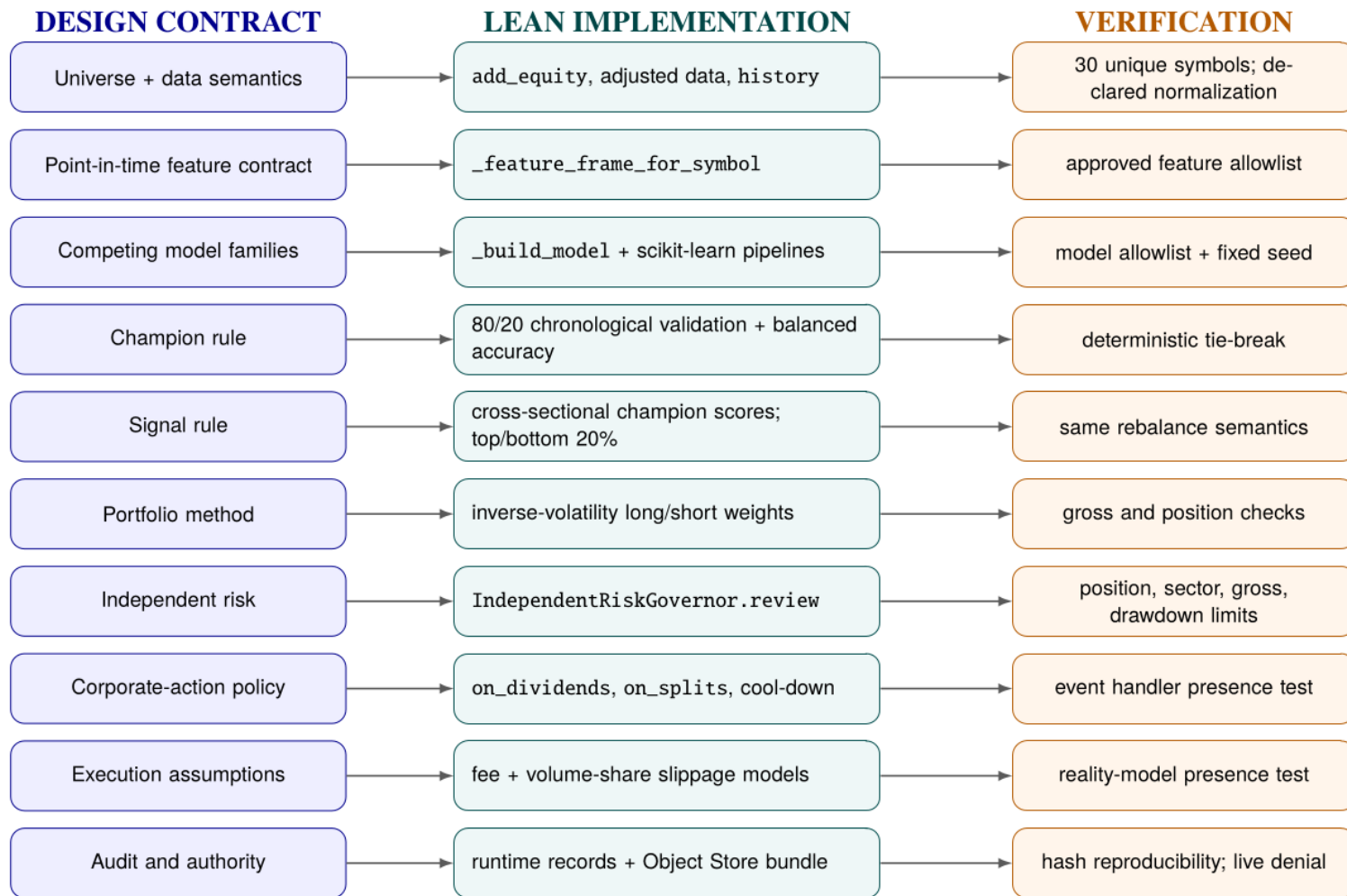


Figure 3: Design-to-implementation mapping. Every major research contract has a LEAN representation and a verification mechanism.

This mapping is the practical meaning of semantic compilation. The universe contract becomes `add_equity` subscriptions and a declared adjusted-data mode. The feature contract becomes a trailing feature function. The model contract becomes `_build_model`. The champion rule becomes the chronological validation tournament. The signal rule becomes cross-sectional ranking. The portfolio rule becomes inverse-volatility weighting. The independent risk contract becomes `IndependentRiskGovernor.review`. Corporate-action semantics become `on_dividends` and `on_splits`. Audit semantics become runtime records and Object Store persistence.

For each mapping, reviewers should ask where a value originates, where it is validated, where it affects runtime behavior, and where evidence is retained. The training cadence originates in the specification, is checked by the compiler, becomes a scheduled callback, and appears in timestamped training records. The same chain applies to universe membership, feature windows, selection fractions, exposure targets, and risk thresholds. It helps locate whether a surprise arose before compilation, during execution, or in reporting.

Mappings can be many-to-one. Several specification fields may be enforced in one risk method, while a single universe declaration affects subscriptions, sector lookup, history requests, and liquidation behavior. These relationships matter because a seemingly local change can have several downstream consequences. At the same time, duplicated rules across unrelated methods should be minimized because duplicate enforcement can drift and become contradictory.

Deterministic names and normalized serialization strengthen traceability. Stable model, feature, and symbol identifiers can appear consistently in configuration, logs, tables, and audit bundles. Stable representation makes diffs meaningful: the reviewer sees the changed assumption rather than noise from key ordering. It also permits automated checks that each approved feature is implemented and each implementation branch corresponds to an approved capability.

The map is therefore a lightweight assurance case connecting intent to code and code to evidence. As the strategy grows, it can become a formal test matrix containing a requirement identifier, implementation reference, test case, result, and reviewer sign-off. The current document establishes that stronger process in conceptual form.

The mapping should also be maintained when code is refactored. A renamed method or reorganized module does not necessarily change semantics, but the documentation and verification references must follow it. Otherwise traceability decays even while the algorithm still runs. Periodic review of the mapping guards against configuration fields becoming vestigial, tests pointing to obsolete behavior, or new runtime branches escaping the original control structure. In that sense, traceability is a maintained property of the project rather than a diagram produced once for publication.

The map can also support change-impact analysis before implementation begins. When a researcher proposes a new field, reviewers can identify every affected runtime component, test, and evidence record. This reduces the chance that a feature addition updates training but not inference, or that a new risk limit appears in configuration without a corresponding audit reason. Explicit impact analysis is particularly valuable when generated code evolves across platform API versions, because it separates necessary mechanical adaptation from unintended changes to the economic experiment.

8 Implementation Layer I: Universe and Data Semantics

The generated strategy uses a stable teaching universe of thirty large U.S. equities divided into six groups: Technology, Financials, Healthcare, Industrials, Consumer, and Energy. Each group contains five securities. This creates sufficient cross-sectional breadth for model ranking and allows the risk governor to express a sector-gross constraint while remaining small enough for transparent inspection.

The current universe is not presented as a production-grade point-in-time institutional universe. The notebook explicitly lists survivorship bias as a known limitation. A production extension should make universe membership itself a governed historical process, including point-in-time eligibility, delistings, liquidity constraints, and security-master changes.

Each equity is added at daily resolution using adjusted data. This is an economically important choice because the historical price series from which returns, momentum, and moving averages are computed depends on the normalization convention. The specification therefore treats ADJUSTED as part of the strategy semantics rather than as an implementation default. Historical observations are requested through LEAN's history interface, which makes the retrieval relative to the current simulation time rather than importing the original notebook database directly [1].

SPY is added separately and used as the benchmark. The benchmark is not part of the thirty-security trading universe.

Data semantics begin before the first feature. Adding a security establishes resolution, normalization, and symbol identity. The algorithm uses adjusted daily data so that historical price relationships remain comparable across splits and distributions, while corporate actions are received as events for eligibility and audit. History requests and runtime slices must follow the same convention. Mixing adjusted training history with raw runtime prices would change returns, moving averages, and volatility in ways easily mistaken for model behavior.

The thirty-name basket is partitioned into sectors that support diversification and the sector-gross control. This metadata is operational state, not decoration. Misspelled symbols, duplicate membership, or an unclassified security can affect subscriptions and aggregation. Compilation should ensure uniqueness, valid assignment, and consistency between the universe and sector map. Runtime should treat unavailable or nontradable securities as ineligible rather than substituting an unauthorized symbol.

Daily resolution establishes a specific information and execution regime. Signals use completed daily observations and scheduled events; the approach makes no claim to intraday timing precision. Holidays, listings, suspensions, and sparse data can leave securities with unequal histories. Eligibility checks must require enough valid observations for all features and must never fill missing history with future or cross-security information.

The static universe creates survivorship and selection concerns because its membership is chosen in advance. That is acceptable for an implementation bridge when stated plainly, but it narrows inference. An institutional extension needs point-in-time membership, delisting treatment, liquidity tests, and historical constituent data. Until then, conclusions concern behavior on the declared basket, not universal equity-selection skill.

Benchmark separation also avoids accidental contamination. SPY can provide a familiar performance reference without becoming an implicit feature or tradable member. If future research uses market returns as predictors or hedges, that role must be declared explicitly because it changes both the information contract and the portfolio. The present distinction keeps benchmark measurement separate from signal generation and capital allocation.

9 Implementation Layer II: Point-in-Time Feature Engineering

For security i at date t , let $P_{i,t}$ denote adjusted closing price and $V_{i,t}$ daily share volume. The one-day return is

$$r_{i,t}^{(1)} = \frac{P_{i,t}}{P_{i,t-1}} - 1.$$

Five-day and twenty-day momentum are

$$m_{i,t}^{(5)} = \frac{P_{i,t}}{P_{i,t-5}} - 1, \quad m_{i,t}^{(20)} = \frac{P_{i,t}}{P_{i,t-20}} - 1.$$

Short- and medium-horizon volatility are rolling standard deviations of daily returns:

$$\begin{aligned} \sigma_{i,t}^{(20)} &= \text{Std}(r_{i,t-19}^{(1)}, \dots, r_{i,t}^{(1)}), \\ \sigma_{i,t}^{(60)} &= \text{Std}(r_{i,t-59}^{(1)}, \dots, r_{i,t}^{(1)}). \end{aligned}$$

Volume enters through a twenty-day z-score,

$$Z_{i,t}^V = \frac{V_{i,t} - \bar{V}_{i,t}^{(20)}}{s_{i,t}^{V,(20)}},$$

and dollar volume $DV_{i,t} = P_{i,t}V_{i,t}$ is standardized analogously,

$$Z_{i,t}^{DV} = \frac{DV_{i,t} - \bar{DV}_{i,t}^{(20)}}{s_{i,t}^{DV,(20)}}.$$

Finally, the feature set measures price relative to the twenty- and fifty-day moving averages:

$$MA20Ratio_{i,t} = \frac{P_{i,t}}{MA_{20,i,t}} - 1, \quad MA50Ratio_{i,t} = \frac{P_{i,t}}{MA_{50,i,t}} - 1.$$

The vector supplied to the model laboratory is therefore

$$X_{i,t} = [r^{(1)}, m^{(5)}, m^{(20)}, \sigma^{(20)}, \sigma^{(60)}, Z^V, Z^{DV}, MA20Ratio, MA50Ratio]_{i,t}.$$

The prediction label is one day ahead. The continuous regression target is

$$y_{i,t}^R = r_{i,t+1},$$

and the classification target is

$$y_{i,t}^C = \mathbf{1}[r_{i,t+1} > 0].$$

The implementation constructs features security by security and then pools observations into a common panel. The ticker is retained for provenance but is not itself a predictive feature. Consequently, the current system learns one global cross-sectional mapping $f(X)$ rather than thirty security-specific models. This increases the effective sample size while assuming that at least some feature-return relationships generalize across securities.

Point-in-time correctness requires more than shifting the label. Every rolling statistic must use only bars complete when the decision is made. Momentum must specify its exact observation span and whether the current completed

bar is included. Volatility must specify return definition, window, and missing-value treatment. Moving-average ratios must use the same adjusted series, while volume features need rules for zero and missing values. Small convention differences can materially alter rankings.

The builder must emit one canonical feature order because estimators consume arrays without necessarily retaining labels. Training and inference use the identical order, which belongs in configuration and diagnostics. Scaling is model-dependent: distance and neural models are sensitive to magnitude, so their transformations must be fitted on training rows only and then applied to validation and current observations. Fitting a scaler on the full panel would leak later distributional information.

Pooling securities increases sample size and asks one estimator to learn relationships that generalize across names. Yet rows on a common date are correlated and securities with longer histories may contribute more observations. A chronological split blocks direct training on later rows, but it does not make samples independent. Scores are comparative evidence within the declared design, not exact confidence measures.

Robust handling favors exclusion over silent imputation. A security lacking the longest lookback, containing nonfinite values, or subject to cool-down should leave the eligible cross section, with a logged reason. Future versions may add governed imputation, winsorization, or neutralization, but each transformation must enter the same timing contract and verification discipline.

10 Implementation Layer III: The Five-Model Laboratory

The predictive system is intentionally heterogeneous. Logistic regression provides a simple linear probabilistic baseline. KNN represents local similarity and nonparametric neighborhood reasoning [2]. Random Forest captures nonlinear thresholds and interactions through a tree ensemble [3]. The MLP provides a compact neural nonlinear function approximator. Linear regression predicts a continuous next-day return and then converts its sign to the common directional validation target. The collection therefore spans linear, neighborhood-based, ensemble, and neural function classes rather than treating one modeling family as universally dominant.

The generated implementations are deliberately bounded:

- Logistic regression is wrapped in a standardization pipeline and uses up to 500 iterations with the declared random seed.
- KNN is standardized, uses $k = 9$, and applies distance weighting.
- Random Forest uses 160 trees, maximum depth 6, minimum leaf size 12, a fixed seed, and one processing thread.
- The MLP uses hidden layers of 24 and 12 units, early stopping, a maximum of 180 iterations, and the fixed seed.
- Linear regression is standardized and trained on the continuous one-day-ahead return.

The project therefore does not encode the proposition that “more complex is better.” Complexity is allowed to compete, but must earn selection through the declared validation criterion.

Comparability requires a common prediction interface although the estimators solve different native problems. Classifiers can provide probabilities or class scores, whereas linear regression produces a continuous return estimate. The portfolio needs an ordering, so each output becomes a scalar whose higher value consistently means a stronger expected outcome. Validation uses a declared class decision so that balanced accuracy has the same target interpretation. Each model adapter must make that conversion explicit.

Hyperparameters are part of the experiment. Neighbor count, forest depth and tree count, neural architecture and iteration limit, regularization, solver behavior, and random seeds can materially change results. They should not be tuned implicitly on the same segment used to select the champion. Here they are fixed and inspectable. A broader search would require nested validation or another governed protocol to avoid hiding selection bias behind automation.

Training failures are evidence. A candidate may encounter a single-class sample, too few observations, convergence problems, or nonfinite input. The tournament should record the failure and continue only when remaining eligible models satisfy the contract. It must not substitute an unapproved estimator or report a failed model as successfully trained.

Heterogeneity does not guarantee diversified errors. Several models may learn nearly identical rankings from the same features, and champion rotation need not prove distinct economic regimes. Score correlations, prediction breadth, turnover caused by model changes, class balance, and stability across nearby windows provide useful context. The laboratory identifies the best candidate under one declared recent historical comparison; it does not identify a universally superior forecasting method.

Finally, candidate complexity should be considered relative to the available sample and feature dimension. More flexible estimators can fit interactions, but they can also exploit transient noise. Fixed seeds and constrained hyperparameters improve reproducibility without eliminating overfitting. The chronological tournament, stability diagnostics, and later falsification tests work together to ensure that flexibility must survive several forms of scrutiny before its predictions influence governed exposure.

11 Implementation Layer IV: Monthly Training and Deterministic Champion Selection

Model training runs through LEAN's training mechanism on the first trading session of each month, sixty minutes before the market open. QuantConnect provides a dedicated training facility for computationally intensive model fitting and scheduled retraining [1].

At a training event, the algorithm requests 320 daily historical bars for the thirty securities, reconstructs the feature panel, and requires a minimum of 500 pooled rows before proceeding. The dataset is sorted in event-time order and then divided by row count into an 80% fitting segment and a 20% validation segment:

$$\mathcal{D}_{train} = \{1, \dots, \lfloor 0.8N \rfloor\}, \quad \mathcal{D}_{valid} = \{\lfloor 0.8N \rfloor + 1, \dots, N\}.$$

Classification models are fit on y^C . Linear regression is fit on y^R , but its validation predictions are converted into direction:

$$\widehat{y}_{i,t}^C = \mathbf{1}[\widehat{r}_{i,t+1} > 0].$$

All candidates are then evaluated through balanced accuracy:

$$BA = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right).$$

The champion is the candidate with the highest balanced accuracy. If scores tie, the code uses the predefined order in `MODEL_FAMILIES`; in implementation terms this is encoded as an enumerated `complexity_rank` and the winner is chosen by sorting $(score, -rank)$ in descending order.

Formally,

$$m_t^* = \arg \max_{m \in \mathcal{M}} (BA_m, -rank_m).$$

No LLM participates in this selection. The winner is a mathematical property of the current training sample, validation segment, candidate library, and tie-break rule.

The champion can change from month to month. A hypothetical sequence could be Random Forest in January, logistic regression in February, KNN in March, and Random Forest again in April. The stable strategy object is therefore the governed model-selection procedure, not any single estimator.

Monthly retraining is a deliberate compromise among adaptation, sample size, turnover, and computational cost. Training every day would cause small data changes to trigger frequent model replacement and would multiply opportunities for validation noise to influence the portfolio. Training too rarely could leave the champion insensitive to material changes. The monthly cadence is therefore a governed experimental choice, not a claim that calendar months are economically optimal. Alternative schedules should be compared through new specifications rather than changed informally.

Chronology is maintained at both the row and event levels. Training observations are ordered so that the validation segment follows the fitting segment, and the scheduled training callback completes before the portfolio asks for champion scores. The forward return label consumes the next daily observation in the historical panel, so the most recent unlabeled row cannot be used as a completed training example. At runtime, current features are used only for inference. These distinctions prevent the convenient but invalid practice of training on the outcome that the same decision is meant to predict.

Balanced accuracy is chosen because the sign of next-day returns may not be evenly distributed. Ordinary accuracy can reward a model for persistently predicting the majority class, whereas balanced accuracy averages performance across positive and nonpositive outcomes. It still has limitations: it ignores probability calibration, economic magnitude, turnover, and trading cost. In this implementation it is a transparent tournament rule, not a complete measure of investment value.

Tie breaking is specified in advance. Without a fixed order, equal rounded scores could make selection depend on dictionary ordering, library behavior, or incidental execution details. The deterministic priority ensures that identical evidence produces the same champion. The training record should include sample sizes, validation dates, each candidate score, failures or warnings, the selected identifier, and the configuration hash. These details let reviewers distinguish a genuine change in evidence from an implementation inconsistency.

The champion remains valid only while its fitted object and required preprocessing state are available. If no candidate trains successfully, or if the validation set is invalid, the safe behavior is to withhold a new portfolio rather than reuse an undocumented fallback. A later system may permit a governed incumbent-retention rule, but that rule would need an explicit maximum age and audit trail. Model freshness is therefore another state variable that should be visible to risk and review.

12 Implementation Layer V: From Champion Scores to a Cross-Sectional Signal

At each portfolio reconstruction, the champion scores the latest feature vector for every eligible security. For probabilistic classification models the score is centered around indifference:

$$s_{i,t} = P(y_{i,t+1} = 1 \mid X_{i,t}) - 0.5.$$

If a classifier does not expose `predict_proba`, the code falls back to its hard class prediction minus 0.5. For the regression champion,

$$s_{i,t} = \widehat{r}_{i,t+1}.$$

The score is used primarily as an ordinal variable. Eligible securities are sorted from weakest to strongest. With N eligible securities,

$$N_L = \max(1, \lfloor 0.20N \rfloor), \quad N_S = \max(1, \lfloor 0.20N \rfloor).$$

The highest-scoring N_L securities form the long candidate set and the lowest-scoring N_S securities form the short candidate set. With all thirty names eligible, this produces six longs and six shorts.

This layer matters because the model target and the portfolio objective are not identical. The models are validated on one-day direction, while the economic strategy uses those scores as a cross-sectional ranking signal and reconstitutes the portfolio monthly. The bridge therefore preserves a distinction between predictive accuracy and investable performance: a good classifier is not automatically a good portfolio, and a useful ranker need not be an extraordinarily accurate directional classifier.

Scores are relative research outputs rather than direct return guarantees. Their principal use is to order the eligible securities at the reconstruction time. Cross-sectional ranking reduces dependence on the absolute scale differences among model families: a regression estimate of 0.002 and a classifier probability of 0.56 cannot be compared economically in isolation, yet each can establish which securities its fitted model prefers. The adapter ensures that ascending and descending order always have the same interpretation.

Eligibility is evaluated before ranks are converted into positions. A symbol may be removed because of insufficient history, nonfinite features, lack of tradability, or a recent dividend or split event. The remaining cross section can therefore vary from month to month. The selection count is derived from the eligible breadth, with explicit rounding and minimum-size rules. If the cross section is too small to form disjoint long and short groups, the algorithm should decline to construct the intended portfolio rather than allow overlap or silently change the fraction.

Approximately the highest twenty percent become long candidates and the lowest twenty percent become short candidates. This rule creates a sparse portfolio and reserves the middle ranks as a no-position zone. The no-position zone is useful because small score differences near the center need not create trades. Nevertheless, rank cutoffs can be unstable: a tiny score change around a boundary can add or remove a security. Diagnostics should therefore retain scores, ranks, cutoffs, selected membership, and changes from the prior selection.

The signal layer does not decide capital allocation. It supplies direction and membership to the portfolio constructor. This separation allows future research to change the weighting rule without retraining the predictive models, or to compare score-weighted and optimized portfolios under the same forecasts. It also prevents a model with unusually large numerical outputs from automatically creating greater gross exposure. The portfolio and risk contracts, not the magnitude of an estimator's raw output, govern capital.

Cross-sectional interpretation should remain modest. A top-ranked security is preferred relative to the eligible set at that moment; it is not necessarily forecast to have a positive absolute return. Likewise, a bottom-ranked security can rise while still underperforming its peers. The long/short construction expresses a relative-value hypothesis, and its results should be evaluated using spread behavior, side-specific returns, turnover, and exposures rather than only individual forecast classification.

13 Implementation Layer VI: Portfolio Construction

The compiled portfolio method is inverse volatility. Within the selected long set L and short set S , preliminary weights are proportional to the inverse of twenty-day volatility. The long side is normalized to approximately +0.5 gross exposure:

$$w_i^L = 0.5 \frac{\sigma_i^{-1}}{\sum_{j \in L} \sigma_j^{-1}},$$

and the short side to approximately -0.5:

$$w_i^S = -0.5 \frac{\sigma_i^{-1}}{\sum_{j \in S} \sigma_j^{-1}}.$$

Before independent risk review, the intended gross and net exposures are therefore approximately

$$\sum_i |w_i| \approx 1.0, \quad \sum_i w_i \approx 0.$$

The signal layer decides *which* securities are preferred; the portfolio layer decides *how much capital* each selected security receives. The separation is conceptually important because it prevents marginal differences in model score from being interpreted mechanically as proportional expected-return estimates.

The notebook also contains alternative equal-weight and volatility-targeting branches. The risk-parity proxy currently shares the inverse-volatility path rather than solving a full covariance-based risk-parity optimization. Likewise, the optional volatility-target branch uses a diagonal approximation based on individual volatilities. These are teaching abstractions, not claims of complete institutional optimization [4].

Inverse-volatility weighting translates selected membership into preliminary exposure while reducing the influence of securities with large recent fluctuations. For each side, the reciprocal of estimated volatility is normalized so that long weights sum to the declared long gross target and absolute short weights sum to the short gross target. The signs are then applied. This produces an approximately dollar-balanced proposal before independent risk review, subject to missing securities and later clamps.

The volatility estimate is itself a governed input. Its return frequency, lookback, normalization, and minimum admissible value must be consistent with the feature and history semantics. Without a floor, a nearly zero estimate could receive an extreme inverse-volatility weight. Without an exclusion rule, nonfinite estimates could corrupt all normalization. A robust constructor therefore validates every estimate, applies only declared safeguards, and refuses to construct a side with no valid denominator.

Separate normalization of long and short sides preserves the intended gross structure but does not ensure beta, sector, factor, or volatility neutrality. A group of low-volatility defensive longs and high-beta shorts can be dollar neutral while retaining substantial market sensitivity. Sector caps in the next layer reduce one form of concentration, but they do not solve general factor exposure. The proposal should consequently be described as gross-balanced rather than broadly neutral.

Portfolio turnover arises from three mechanisms: changes in champion predictions, movement across selection cutoffs, and changes in inverse-volatility weights for continuing names. Rebalancing every selected weight to a new target may create trades even when membership is stable. Useful evidence includes pretrade and posttrade weights, membership additions and removals, target changes, expected turnover, and side-level gross. A production extension could introduce buffers, minimum trade sizes, or turnover penalties, but these would alter the portfolio contract and require explicit evaluation.

The constructor outputs a complete proposed weight map, including zeros for positions that should be exited. This is important in an event-driven engine because omitting an old holding from a new target set does not necessarily

liquidate it automatically. The subsequent execution step should compare approved targets with current holdings and send only the actions required to reach the governed state. Thus portfolio construction defines the desired exposure, while LEAN manages the orders and accounting used to approach it.

14 Implementation Layer VII: Independent Risk as a Veto Layer

The predictive model never writes final portfolio weights directly to the order API. Preliminary weights are sent to `IndependentRiskGovernor.review`. The risk class does not fit models and does not select the champion. Its sole responsibility is to evaluate proposed exposure against current portfolio state and declared limits.

The first control caps each position:

$$|w_i| \leq 0.10.$$

The second control limits gross exposure in each declared sector group S_k :

$$\sum_{i \in S_k} |w_i| \leq 0.35.$$

If a sector exceeds the cap, all bounded weights in that sector are proportionally scaled down.

The third control limits total gross exposure:

$$\sum_i |w_i| \leq 1.00.$$

If necessary, all positions are scaled proportionally.

The fourth control tracks a portfolio high-water mark. Let V_t be current total portfolio value and

$$V_t^{max} = \max_{\tau \leq t} V_\tau.$$

The current drawdown is

$$DD_t = \frac{V_t}{V_t^{max}} - 1.$$

If

$$DD_t \leq -0.12,$$

then the governor enters a halted state and returns an empty approved portfolio.

Because the rebalance procedure subsequently calls

```
set_holdings(symbol, approved.get(symbol, 0.0))
```

for every universe symbol, an empty approved portfolio targets all positions to zero at that rebalance.

The drawdown rule is evaluated when the risk governor reviews a portfolio proposal, and portfolio proposals occur on the monthly rebalance schedule. The current teaching implementation is therefore not a continuously monitored stop. A production architecture should decouple critical risk-monitoring frequency from the portfolio-rebalance frequency.

The current risk separation is also logical rather than service-level. The governor is a separate class but executes in the same `QCAAlgorithm` process. The notebook itself identifies this as a limitation. A production institution could move risk controls into separate supervisory services, account-level controls, or broker-side limits.

Calling the governor independent describes logical authority rather than a separate deployed service. It receives a proposed portfolio and may reduce or reject it without asking the champion model for permission. The governor does not improve forecasts or select securities; it evaluates whether the aggregate proposal fits declared capital

constraints. Keeping its interface at the weight level makes the decision inspectable and prevents risk rules from becoming hidden features of the estimator.

Constraint order matters when several limits bind. Per-position clamps alter sector and total gross; sector scaling alters total gross; total-gross scaling may reduce all positions; a drawdown halt overrides the entire proposal. The implementation should use a deterministic sequence and recalculate diagnostics after each stage. Otherwise the final portfolio might violate a constraint that was checked only before another transformation. The audit record should distinguish proposed weights from each intermediate and final approved state.

The position limit caps absolute weight, treating long and short exposures symmetrically. Sector gross sums absolute exposures within each sector so that offsetting a long and short in one group does not conceal concentration. Total gross similarly sums absolute weights rather than netting sides. These definitions reflect financing and loss potential more directly than net exposure, although they do not measure liquidity, beta, or nonlinear risk.

The drawdown control compares current portfolio value with the running peak. Once the declared threshold is breached, the governor can return zero targets and prevent ordinary reconstruction from immediately reestablishing exposure. Precise restart behavior matters. An institutional design would specify whether the halt is permanent for the run, lasts for a cooling period, or requires human acknowledgment. The bridge uses a conservative veto concept, and reviewers should confirm its realized state transitions in logs.

Risk limits cannot guarantee safety. Gaps, unavailable liquidity, correlated moves, delayed events, model error, and fill uncertainty can cause realized exposure or loss to exceed an intended bound. The governor controls target weights under simulated information; LEAN's fills determine achieved holdings. Verification must therefore compare proposed, approved, ordered, and filled states. A difference is not automatically a defect, but it must be visible and reconciled.

Independent risk also improves research interpretation. If performance changes because a sector cap scales the portfolio, the effect can be attributed to a documented control instead of being confused with predictive quality. Researchers can separately report raw signal behavior, unconstrained portfolio behavior, and governed results. This prevents strong forecasts from being credited for leverage they were never authorized to take and prevents risk reductions from being mislabeled as model failures.

15 Implementation Layer VIII: Corporate Actions and Eligibility

In the research notebooks, corporate actions can be represented as records in a controlled dataset. In LEAN, they are events. NB07 therefore records dividends and splits through `on_dividends` and `on_splits`; LEAN's event-handler model is designed precisely for such sequential processing [1].

Whenever a dividend or split is received for a tracked security, the algorithm stores the current calendar date. During portfolio construction, `_recent_corporate_action` excludes a security when the current date minus the stored event date is less than or equal to the configured two-day cool-down.

Two details are worth making explicit. First, the current rule uses calendar-day differences, not a count of exchange sessions. Second, exclusion affects the portfolio proposal; because all securities absent from the approved portfolio are targeted to zero at rebalancing, the practical effect is not merely “do not initiate a new position.” A security under cool-down can be liquidated at the next monthly reconstruction.

The information doctrine is stronger than the cool-down policy. Future corporate-action knowledge is never a predictive feature. The platform event may affect eligibility only after it has actually been delivered in simulated time.

Corporate actions create both economic adjustments and operational events. Adjusted historical prices are intended to preserve comparability across distributions and splits, but the event stream still matters because an action can coincide with discontinuities in volume, tradability, metadata, or feature interpretation. Recording dividends and splits therefore serves a different purpose from price normalization. One affects the data series used for calculation; the other informs runtime eligibility and evidence.

The two-calendar-day cool-down is a conservative rule that temporarily removes an affected symbol from new ranking and selection. Calendar days are not identical to trading sessions, so a Friday event and a midweek event can imply different numbers of intervening market bars. The implementation should record the event timestamp and computed expiry so that reviewers can confirm the intended convention. A future specification may prefer exchange sessions, but that would be a semantic change.

Eligibility affects both entries and existing positions. If an already held security enters cool-down, the desired response must be explicit: maintain, reduce, or liquidate. The safe interpretation in a complete target-weight process is that an ineligible symbol is absent from the approved active set and receives the contractually defined target, commonly zero. This behavior should be verified from orders rather than inferred from feature exclusion alone.

Multiple events can occur close together. A new split or dividend should extend or replace the cool-down according to a deterministic maximum-expiry rule, not shorten an existing restriction. Event records should include symbol, type, relevant quantities, observation time, prior expiry, and new expiry. These details support reconciliation with adjusted history and make unusual post-event trades explainable.

Corporate-action handling remains simplified. Real institutional treatment can include mergers, spin-offs, symbol changes, rights, special distributions, delistings, and tax or borrow effects. The bridge does not claim comprehensive lifecycle processing. Instead, it demonstrates how such policies should enter: as explicit event handlers, eligibility state, auditable decisions, and governed extensions rather than ad hoc exceptions inside the forecasting code.

16 Implementation Layer IX: Fees, Slippage, and Execution Reality

Each subscribed security receives a `ConstantFeeModel` and a `VolumeShareSlippageModel`. This is the platform-native counterpart of the broader project's insistence that execution costs remain explicit rather than being silently ignored.

The current fixed-fee calibration is

$$\text{PER_ORDER_FEE_USD} = 0.00.$$

Accordingly, the architecture is fee-aware, but the present default does not impose a positive commission. Empirical use should therefore calibrate or stress this parameter according to the brokerage context.

The volume-share slippage parameters are

$$\text{VOLUME_SHARE_LIMIT} = 0.025, \quad \text{VOLUME_SHARE_PRICE_IMPACT} = 0.10.$$

A volume-share model links simulated impact to order size relative to observed market volume, making the execution assumption sensitive to capacity rather than merely subtracting a constant return penalty [1].

The architecture does not claim that this one calibration is universally realistic. Its methodological claim is narrower and more important: the cost model must be visible, configurable, and included in falsification. If an apparent edge disappears under plausible cost and slippage ranges, the research process should reject the candidate rather than hide the sensitivity.

Reality models are part of the tested strategy because they influence the conversion from targets to simulated fills. A constant fee of zero makes the current calibration transparent, but it does not make costs economically absent. It should be interpreted as a placeholder that isolates other implementation effects. Before drawing practical conclusions, reviewers should rerun the experiment with plausible commissions, regulatory charges, spread costs, borrow fees, and market impact.

The volume-share slippage model links simulated price impact to order participation in observed volume, subject to its configured limit and coefficient. This is more informative than assuming every order fills at an unmodified reference price, yet it remains an approximation. Daily bars do not reveal intraday queue position, spread evolution, or the path by which an order consumes liquidity. The model's parameters should therefore be treated as scenario assumptions and tested over conservative ranges.

Order sizing interacts with slippage. A target weight becomes a quantity using current portfolio value and security price; rounding and existing holdings change the actual order. Several securities rebalanced together may create simultaneous gross turnover not apparent from a single order. Evidence should report target change, submitted quantity, reference price, fill price, fees, and estimated participation where available. Aggregating these records by side, sector, and rebalance helps explain the gap between forecast returns and realized simulated returns.

Short positions add costs and constraints that the basic model does not fully represent. Borrow may be unavailable, recalls may occur, and borrow rates can vary sharply. A cloud backtest that permits a short fill is not proof that the trade would have been available at the assumed economics. A production-oriented extension should incorporate borrow eligibility, locate policy, hard-to-borrow costs, and stress scenarios.

Execution realism should be evaluated through sensitivity rather than one preferred calibration. If modest changes in impact, delay, or fees eliminate the result, the strategy is economically fragile even if the baseline curve is attractive. Conversely, stable behavior across conservative assumptions strengthens, but does not prove, implementability. The governed architecture supports these comparisons because execution parameters are explicit and can define distinct hashed experiments.

17 Runtime Scheduling and Event-Driven Execution

The generated class `GovernedQuantConnectBridge(QCAlgorithm)` starts by enforcing the authority boundary: if the algorithm is in live mode, or if configuration flags indicate live or deployment authority, initialization raises an exception. The algorithm then sets sample dates, starting cash, benchmark, state variables, event-date storage, the model champion state, and the independent risk governor.

The thirty securities are subscribed, sector membership is stored, fee and slippage models are attached, and an 80-bar warm-up is declared. Model training is registered through LEAN's `train` mechanism on the monthly start date before market open. Portfolio reconstruction is registered through `schedule.on` for thirty minutes after the market opens on the same monthly schedule.

The runtime sequence is therefore

Initialize → Warm Up → Monthly Train → Champion Selection
→ Monthly Rebalance → Risk Review → Orders / Fills → Audit.

This temporal separation is important. The model tournament is completed before the portfolio asks the champion to score current securities. Training and portfolio construction are separate scheduled responsibilities rather than one opaque event function.

An event-driven algorithm is a state machine. Initialization declares securities, models, schedules, reality assumptions, and persistent state. Data and corporate-action events update that state. Training callbacks fit candidate models and select a champion. Reconstruction callbacks request features, produce proposals, obtain risk approval, and submit targets. Order events report the platform's response. Termination serializes evidence. Understanding this sequence is essential because the same calculations performed in a different order can use a different information set.

Scheduled training and reconstruction must have an explicit relative order. If both occur on the same date, the portfolio should either use the newly trained champion or the incumbent according to the declared contract. Reliance on incidental callback registration order is fragile. Timestamps in audit records should show which model version generated each rebalance. Similarly, warm-up completion must precede actions that require full trailing history.

Not every scheduled event will yield a trade. History may be insufficient, the champion may be unavailable, all symbols may be ineligible, the selection set may be too small, or risk may veto exposure. These no-action outcomes are meaningful and should be logged with reason codes. Silence makes it impossible to distinguish an intentional skip from a failed callback.

LEAN owns exchange calendars, sequential data delivery, security state, order tickets, fills, and portfolio accounting. The algorithm should use those services rather than simulate a second independent clock inside the strategy. This reduces inconsistencies around holidays and tradability. Nevertheless, schedules defined relative to market open or close require careful thought about which daily bar is complete at the callback time.

Event-driven verification focuses on temporal invariants: no training row uses a future label, no score is generated before its model is fitted, no risk-approved target bypasses review, and no terminated run omits its audit save attempt. These invariants can be inspected through ordered logs and small controlled backtests. They are more informative than checking only whether the final equity curve exists.

Because callbacks may be retried or encountered under unusual platform state, idempotence is desirable where practical. Repeating an audit write should not create ambiguous identities, and repeating a target calculation from unchanged state should yield the same proposal. Order submission requires greater care because duplicate submissions have financial meaning. Unique event identifiers and recorded model versions help detect accidental repetition.

18 Falsification Before Performance Claims

The generated project is itself treated as a hypothesis that can fail before any Sharpe ratio is considered. The local falsification matrix checks that the generated Python parses, the `StrategySpec` contains every required field, the universe has thirty unique securities, all models and features are allowlisted, the chosen portfolio method is approved, promotion remains human-controlled, live and deployment authority are denied, the independent risk class exists, deterministic champion selection is present, corporate-action handlers exist, cost models are present, runtime audit machinery exists, no LLM client is embedded in `main.py`, no prohibited network code appears, and repeated serialization yields the same project hash.

This layer does not prove investment quality. It answers a more primitive question:

Did we compile the research institution that we said we intended to compile?

Only after that question is answered does platform performance evidence become meaningful.

The experiment registry reinforces the distinction. The notebook records

```
status = READY_FOR_QUANTCONNECT_BACKTEST,
platform_compile_status = NOT_RUN,
platform_backtest_status = NOT_RUN.
```

The research system refuses to manufacture downstream evidence merely because that evidence is expected to exist later.

Falsification asks what observation would cause the research claim to be weakened or rejected. For this bridge, the first targets are semantic: feature values should match independently calculated examples, schedules should use the intended completed bars, eligible symbols should match policy, and approved weights should satisfy every limit. A strategy that fails these tests should not proceed to performance interpretation, even if its returns look favorable.

Statistical falsification then challenges the model tournament. Results should be compared across nearby training windows, alternative chronological splits, different seeds where applicable, and subperiods with different market conditions. Champion scores should be checked against simple baselines such as majority-class prediction, fixed linear models, and randomized labels. If apparent skill disappears under small reasonable perturbations, it may reflect validation noise or selection luck.

Economic falsification challenges the portfolio. Researchers should separate long-side, short-side, and spread returns; examine turnover and concentration; apply conservative fees, impact, and borrow assumptions; and test delays in acting on scores. Factor and sector attribution can reveal whether the strategy is primarily harvesting an undeclared market exposure. Capacity analysis asks whether the order sizes remain plausible relative to volume.

Operational falsification challenges the implementation under awkward events: insufficient history, missing bars, model convergence failures, equal validation scores, corporate actions, halted securities, rejected orders, and drawdown breaches. Expected safe behavior should be specified before testing. A graceful zero portfolio with a clear reason can be a successful operational result, whereas an unexplained trade is a failure even if profitable.

No single falsification test proves validity. The objective is to create multiple opportunities for a weak hypothesis or flawed translation to reveal itself before promotion. Negative findings should remain in the experiment record rather than being discarded while parameters are adjusted. This preserves the path by which the final design was selected and reduces the risk of presenting adaptive trial and error as one prespecified success.

The order of testing should favor cheap, decisive checks first. Static contract validation and unit examples can reject defects before a long backtest. Short diagnostic periods can exercise event sequencing. Only then should computationally expensive full-history runs and parameter sensitivities proceed. This staged approach conserves research attention and reduces the temptation to rationalize a flawed implementation after seeing an attractive result.

19 Audit, Provenance, and Object Store Evidence

Stable SHA-256 hashes are used for deterministic artifacts. The compiled specification has a strategy-specification hash; the generated project has an artifact hash; the experiment registry has its own registry hash. During runtime, the `_record` method stores the current time, event type, and payload, then attaches a stable hash to the resulting event record.

Material event types include initialization, skipped or failed training, champion selection, portfolio rebalance, dividends, splits, and order events. At the end of the algorithm, a bundle is created containing the final champion, model diagnostics, last risk report, all audit events, an aggregate audit hash, and a configuration hash. The bundle also repeats that live trading and deployment are unauthorized.

The bundle is saved to the QuantConnect Object Store under the configured key `NB07/audit_bundle.json`. Object Store persistence provides a platform-native mechanism for retaining algorithm artifacts beyond the event stream [1].

The current construction should be described precisely as per-record hashing plus a terminal hash of the audit collection. It is not yet a cryptographic hash chain in which each record commits explicitly to the prior record. A production hardening exercise could add that stronger structure if required.

Audit evidence answers who or what made a decision, under which configuration, from which inputs, at what simulated time, and with what result. The bundle should therefore include the strategy and compiler identifiers, configuration hash, platform run context, model-training diagnostics, champion changes, eligibility events, proposed and approved portfolios, risk actions, order events, and termination status. Where the platform supplies identifiers, they should be retained to connect algorithm records with engine records.

Provenance begins before runtime. The source specification, validation report, generated configuration, generated algorithm, and experiment registry collectively describe how the runnable project came to exist. A hash is most useful when its canonicalization procedure is stable and the hashed artifact is also preserved. Otherwise a future reviewer may know that two runs differed without being able to reconstruct why.

Object Store persistence gives the algorithm a platform-native location for its terminal evidence. Saving at termination is useful, but the design should also consider partial failure. A crash before the final callback could lose in-memory records. An institutional extension could checkpoint important events, limit record size, and verify successful writes. Storage errors themselves should be surfaced because an unaudited run should not be treated as equivalent to a fully evidenced run.

The current hashes do not form a predecessor-linked cryptographic chain, so they support identity and tamper indication only within their stated scope. Stronger designs could chain records, sign bundles, separate write authority, and retain immutable external copies. These controls matter more when audit artifacts influence approval or regulatory obligations.

Audit is not synonymous with logging everything. Excessive unstructured logs can conceal important transitions and make retention expensive. The preferred design combines concise typed records for governed decisions with selected diagnostics for investigation. Each record type should have a stable schema, timestamps, identifiers, and a documented retention purpose. The result is evidence that can be queried and compared, not merely a large text transcript.

20 Semantic Reconciliation: Why the Notebook and LEAN Need Not Produce Identical Curves

A common mistake in strategy migration is to treat numerical identity between a vectorized notebook backtest and an event-driven platform as the only acceptable definition of correctness. NB07 instead defines a reconciliation contract.

Certain objects should match tightly: the universe size, feature contract, model families, selected portfolio method, maximum gross exposure, maximum position size, event-cool-down policy, and authority state. Other quantities should be compared with tolerance and interpreted through platform semantics. Turnover may change because LEAN produces actual order and fill sequences. Transaction costs are governed by LEAN's reality models. Equity and drawdown are functions of LEAN portfolio accounting. Event timing and eligibility are functions of the platform calendar.

The correct standard is therefore *semantic equivalence*, not naive numerical identity. If the platform equity curve differs, the researcher should ask whether the difference arises from legitimate execution semantics or from a translation defect.

Research code and platform code should agree on what the strategy means. They do not need to pretend that a vectorized table and an event-driven execution engine experience time, fills, accounting, and corporate actions identically.

Reconciliation separates legitimate implementation differences from semantic defects. The notebook may use vectorized orders at an abstract closing price, while LEAN applies scheduled callbacks, quantity rounding, slippage, fees, market calendars, and sequential fills. These mechanics can produce different turnover, realized prices, cash paths, and returns without implying that the strategy contract changed. Such differences should be measured and classified.

Other differences are not acceptable. Universe membership, data normalization, lookback formulas, feature order, label horizon, validation chronology, tie breaking, selection fractions, and risk limits should match the declared semantics. If the notebook includes information unavailable at the LEAN decision time, the notebook result is not a valid reference. If the compiler changes a threshold or ignores a field, the platform result is not a faithful implementation. Reconciliation must be willing to identify either side as defective.

A useful contract compares intermediate artifacts rather than only equity curves. On selected dates, reviewers can compare trailing input bars, feature vectors, training and validation boundaries, candidate scores, champion identity, current cross-sectional scores, selected membership, preliminary weights, risk adjustments, orders, and fills. The first point of divergence usually reveals the relevant layer. Comparing only cumulative return loses this diagnostic advantage because many distinct defects can produce a similar final difference.

Tolerance rules should reflect the quantity. Symbol sets and authority flags require exact equality. Floating-point features may allow small numerical tolerances. Portfolio weights may differ because of explicit rounding, while filled holdings may differ further because of order mechanics. Every tolerance needs a rationale; broad unexplained tolerances can mask semantic drift.

Reconciliation outcomes belong in the experiment registry. A run can compile and finish yet remain unverified if important mismatches are unexplained. Conversely, a different equity curve may be accepted when all intermediate semantics match and the residual difference is attributable to documented platform mechanics. The purpose is not to force cosmetic equality, but to establish that observed behavior arises from the intended governed experiment.

21 What the QuantConnect User Is Actually Running

A reader who wants the algorithm without the entire conceptual history can summarize it as follows.

The project subscribes to thirty U.S. equities at daily adjusted resolution and uses SPY as benchmark. Once per month it retrieves 320 trailing bars and constructs nine return, momentum, volatility, trading-activity, and trend features. It pools those observations across the cross section. Five approved models are fit. The last 20% of the pooled chronological rows act as validation. Balanced directional accuracy determines the champion. Thirty minutes after the monthly open, that champion scores the latest eligible securities. Securities with a sufficiently recent dividend or split are excluded. The cross section is ranked. Approximately the top 20% become longs and the bottom 20% become shorts. The two sides are weighted by inverse twenty-day volatility and normalized toward +50% and -50% gross exposure. Independent risk then caps positions at 10%, caps sector gross at 35%, caps total gross at 100%, and can halt the portfolio following a 12% observed drawdown. Approved targets are sent through `set_holdings`. LEAN handles order events, fills, portfolio state, fee/slippage reality models, and corporate actions. The runtime records an audit trail and saves an evidence bundle at termination.

That is the trading system. The surrounding architecture explains why every stage is separated, bounded, and auditable.

The runnable object is not just a fitted model. It is a scheduled algorithm with subscriptions, rolling-history requests, fitted preprocessing and estimator state, eligibility state, portfolio holdings, risk state, and an audit recorder. At initialization it declares the backtest period, cash, benchmark, universe, data normalization, fee and slippage models, and scheduled activities. Those declarations shape every later observation and therefore form part of the strategy rather than platform decoration.

During a normal training cycle, the algorithm retrieves only history available at the simulated time, reconstructs the pooled panel, fits each approved candidate, measures the chronological validation segment, and stores one champion with its preprocessing pipeline. During reconstruction, it builds current features for eligible symbols, asks that champion for comparable scores, ranks the cross section, constructs inverse-volatility targets, submits them to risk, and passes only approved weights to the platform. Existing positions omitted from the approved target set are assigned exit targets so that stale holdings do not persist accidentally.

The user is also running several safety behaviors. Live mode is rejected. Missing or invalid data can prevent action. Corporate actions can temporarily remove a security. Position, sector, gross, and drawdown limits can reduce or eliminate the portfolio. Model-training or selection failures do not authorize an improvised replacement. These no-trade outcomes are part of the designed algorithm and should be expected in diagnostic logs.

What the user is not running is equally important. The project does not contain an online LLM that changes code or trades during the backtest. It does not dynamically search arbitrary models or features. It does not claim a point-in-time institutional universe, comprehensive borrow model, factor-neutral optimizer, or production deployment service. It does not convert a successful backtest into live authority. The generated files are the bounded endpoint of an upstream research process.

Operationally, the most useful way to inspect a run is to follow one reconstruction date through the records: identify the active champion and training evidence, confirm the eligible universe and feature availability, inspect scores and selected tails, reproduce preliminary weights, verify each risk transformation, and compare approved targets with orders and fills. This trace demonstrates what the QuantConnect user is actually running far more reliably than an algorithm name or a final performance summary.

At the end of the run, the user should possess enough evidence to reconstruct this path without rerunning the algorithm immediately. If champion identity, risk transformations, or order linkage cannot be recovered, the implementation may still have traded in simulation, but it has not met the document's standard of governed inspectability.

22 Why the Best Historical Model Is Not Hard-Coded

Suppose a retrospective experiment over 2021–2025 showed that Random Forest produced the highest terminal Sharpe ratio. A conventional workflow might delete the losing models and publish the Random Forest strategy. The governed architecture deliberately avoids that move because it would use information from the entire backtest to choose the model that appears to have been used prospectively.

Instead, the selection function is part of simulated time. At each training date,

$$m_t^* = \mathcal{S}(\mathcal{M}, \mathcal{D}_{\leq t}, \mathcal{V}),$$

where $\mathcal{M}$ is the approved model library, $\mathcal{D}_{\leq t}$ is information available through time t , $\mathcal{V}$ is the chronological validation protocol, and $\mathcal{S}$ is the deterministic champion rule. Portfolio construction and risk then transform that champion’s current scores:

$$W_t = \mathcal{R} \left[\mathcal{P} \left(m_t^*(X_t) \right) \right].$$

The stable object of analysis is thus not “Random Forest” or “KNN.” It is the entire governed procedure. This is particularly important in machine-learning research, where repeated model and hyperparameter selection can create backtest overfitting even when individual predictions appear formally out of sample [5].

Hard-coding a model would make the strategy definition depend on one historical comparison made before the run. That approach is reasonable when strong prior evidence supports a fixed estimator, but it is not the proposition tested here. This project treats model identity as a periodically selected state under a prespecified tournament. Logistic regression, KNN, random forest, MLP, and linear regression remain eligible candidates, and the current evidence determines which supplies rankings until the next successful training event.

This design recognizes that market relationships and sample properties can change. A linear model may be competitive when effects are weak and stable; a tree ensemble may benefit from nonlinear interactions; a distance model may exploit local structure; and a neural network may fit broader nonlinear patterns. None receives authority because of reputation or complexity. The same features, chronology, and metric constrain their comparison.

Adaptive selection also creates risk. Repeatedly choosing the highest validation score adds a layer of selection noise, especially when candidate scores are close. The champion may rotate because of sampling variation rather than a genuine regime change. That is why deterministic ties, fixed candidates, stable hyperparameters, champion history, and sensitivity analysis matter. Future versions could require a minimum improvement over the incumbent, average evidence across folds, or use ensembles, but those are new rules requiring explicit tests.

The selected model is best only in a narrow conditional sense: it has the highest declared balanced accuracy on the current validation sample among candidates that trained successfully. It is not proven to maximize return, Sharpe ratio, calibration, stability, or performance after costs. The wording in reports should preserve this qualification. Overstating the meaning of “best” is a common source of model risk.

Not hard-coding the winner does not mean removing human control. Humans approve the candidate registry, features, metric, schedule, tie-break order, portfolio method, and risk limits. The algorithm exercises bounded adaptation inside those choices. If evidence suggests that the tournament itself is poorly designed, researchers revise the governed specification and run a new experiment rather than letting the runtime redefine its own objective.

23 Where Artificial Intelligence Enters—and Where It Does Not

AI enters the project in two different senses. The first is predictive machine learning: KNN, Random Forest, and the MLP search for statistical relationships between market features and subsequent returns; the regression models supply simpler competing representations.

The second is research-process intelligence. In the larger architecture, an LLM can read an ordinary-language mission, interpret what the researcher is asking, propose a bounded set of approved model families, identify portfolio alternatives or falsification tests, and criticize research conclusions. That role can dramatically enlarge the hypothesis space and reduce the friction of coordinating many analytical tools.

The LLM nevertheless remains outside the LEAN runtime. The generated `main.py` contains no OpenAI client, and the falsification matrix explicitly checks for this. The planner does not choose the champion after observing scores. It does not call `set_holdings`. It does not modify the risk limits. It does not self-authorize live trading.

The architecture can therefore be summarized as

AI-assisted research reasoning → validated specification → deterministic platform execution,

not

LLM opinion → order.

This is the essential separation of cognition from financial authority.

Artificial intelligence enters upstream as an optional aid to interpretation and research planning. A language model can help convert a natural-language objective into candidate hypotheses, identify missing assumptions, summarize prior experiment evidence, or propose tools from an approved registry. Its output is a proposal that remains subject to schema validation, allowlists, mission constraints, deterministic compilation, and human review. This use exploits broad reasoning without granting the model direct financial authority.

The runtime machine-learning models are also forms of artificial intelligence in the broad sense, but their role is narrower. They fit numerical relationships from the declared panel and emit scores through fixed interfaces. They cannot change the universe, add features, modify risk limits, access unapproved tools, or promote the strategy. Their competition is mathematical and scheduled, not conversational.

No language model is called from LEAN during market events, training, ranking, risk review, or order submission. This exclusion improves reproducibility and avoids dependence on changing remote responses, network availability, or unversioned prompts. It also removes a path by which text could become an order without deterministic validation. Generated source can be inspected before a run and linked to a configuration hash.

Human authority remains necessary at consequential transitions. A person defines or approves the research mission, reviews capabilities, interprets falsification evidence, accepts or rejects reconciliation differences, and decides whether another stage is warranted. Human review is not assumed infallible; it is an accountable authority boundary. Decisions should be recorded with the same care as automated events.

Future AI assistance can expand safely if capabilities remain explicit. An agent might draft tests, compare audit bundles, propose sensitivity grids, or explain failures. It should operate with least privilege, bounded inputs and outputs, retained provenance, and no ability to approve itself. The central rule is that intelligence may enlarge the search and criticism process, while executable permissions continue to come from deterministic contracts and authorized reviewers.

This boundary also protects the language model from being treated as an oracle. Its useful contributions are hypotheses, questions, summaries, and drafts that can be checked. Statements about expected returns, risk, or implementation correctness require numerical evidence and deterministic verification. The system values the model's breadth while explicitly accounting for its capacity to produce plausible but unsupported content.

24 Technical Limitations and Research Extensions

The implementation is intentionally transparent about its current limitations.

24.1 Universe construction

The static thirty-security fixture is pedagogically useful but can contain survivorship bias. Institutional research should replace it with a point-in-time selection process.

24.2 Chronological split by row count

The pooled dataset is sorted by event time but cut at an 80% row boundary. Because multiple securities can share the same date, the split can in principle place some rows from one date in training and other rows from the same date in validation. A stricter implementation should split by complete date blocks and, where necessary, apply purge or embargo procedures.

24.3 Repeated model selection

A single trailing holdout does not eliminate second-order overfitting from repeatedly choosing among model families. Walk-forward validation, nested model selection, and explicit multiple-testing controls are natural extensions [5].

24.4 Champion metric

Balanced accuracy is a reasonable common directional criterion, but it is not necessarily the economically optimal model-selection metric for a cross-sectional portfolio. Future work could compare information coefficient, rank correlation, long-short spread return, turnover-adjusted utility, or other out-of-sample metrics.

24.5 Forecast and holding horizons

The label horizon is one day while the portfolio is reconstructed monthly. This is an intentional but nontrivial design choice. Signal aggregation, forecast horizon, and rebalance frequency should be treated as research dimensions rather than assumed optimal.

24.6 Costs and shorting

The current fixed order fee is zero, while slippage is nonzero. Institutional research should calibrate commissions, borrow availability, stock-loan fees, short-sale constraints, and market impact.

24.7 Risk monitoring frequency

The drawdown control is evaluated through the monthly risk-review path. Critical controls should generally be evaluated more frequently than portfolio reconstruction.

24.8 Factor neutrality

A +50%/-50% gross structure is approximately dollar neutral before risk scaling, but it is not necessarily beta neutral, sector neutral after scaling, or neutral to common factors. A richer portfolio layer could impose explicit beta and factor constraints.

24.9 Audit hardening

The runtime records contain hashes, but not a predecessor-linked cryptographic chain. A stricter audit design could chain records and separate write authority from the trading process.

These limitations are not reasons to discard the architecture. They are the next falsification targets. A good autonomous research system should make its unresolved assumptions easier to identify, not harder.

Several limitations interact. A static survivorship-prone universe can make model validation appear cleaner than a point-in-time universe. A short chronological split can make champion identity unstable. Frequent membership changes can increase turnover, whose cost is understated by a zero-fee calibration. Simplified borrow assumptions can make the short side look more implementable than it is. These concerns should be evaluated jointly rather than checked off independently, because their combined effect can be larger than any isolated adjustment.

The feature set is intentionally narrow and price-volume based. It omits fundamentals, analyst revisions, macroeconomic releases, options information, microstructure, and text. Adding data is not automatically beneficial. Every source introduces publication timing, revision history, identifier mapping, missingness, licensing, and operational dependencies. An extension should document when a datum became observable, not merely the date it describes, and should reproduce that availability in the platform.

The validation method is a simple chronological holdout rather than a full walk-forward or nested protocol. A stronger design could use several anchored or rolling folds, purge observations around boundaries where labels overlap, and reserve a final untouched evaluation period. Statistical uncertainty around balanced accuracy and rank performance should be reported, particularly because pooled rows share dates and common market shocks.

The portfolio method does not model covariance or explicit factor exposure. Extensions may introduce covariance shrinkage, beta and sector neutrality, turnover penalties, liquidity constraints, and robust optimization. Complexity should be justified by incremental falsification evidence, since an optimizer can amplify estimation error and obscure the relationship between scores and positions.

Operationally, the governor is logically independent but colocated with the algorithm, audit records are not cryptographically chained, and terminal persistence can be vulnerable to incomplete runs. Production architecture could separate services and credentials, add immutable event storage, monitor liveness, reconcile broker state, and enforce controls outside the research process. Live deployment would also require secrets management, incident response, change approval, and rollback procedures absent from a research bridge.

These limitations define a prioritized research agenda. The next work should address assumptions most likely to reverse the conclusion: point-in-time universe construction, realistic short and trading costs, walk-forward stability, factor attribution, and execution sensitivity. Only after those tests should broader model or feature searches be treated as valuable. Extending search before repairing the evidence base can increase the number of attractive false discoveries.

25 Practical QuantConnect Installation and Research Workflow

The generated package is intentionally compact. The principal platform files are `main.py` and `strategy_config.py`. The full research package may additionally contain `strategy_spec.json`, `experiment_registry.json`, `validation_report.json`, `reconciliation_contract.json`, and the original research notebook.

A research user can install the bridge into QuantConnect Cloud by creating a Python project, replacing the default `main.py` with the generated version, adding `strategy_config.py` to the same project, and then running a backtest. The source code intentionally raises an error if the project is run in live mode. After the backtest, the user should review model diagnostics, champion history, orders, fills, portfolio exposures, risk reports, corporate-action events, standard QuantConnect backtest statistics, and the Object Store audit artifact.

The reconciliation contract should then be used to classify differences between notebook expectations and platform behavior. A difference in turnover or transaction costs can be a legitimate consequence of LEAN order mechanics; a difference in feature definitions, universe membership, or authority state is more likely to indicate a semantic translation defect.

The experiment registry should not be manually changed to indicate success before the platform run. The correct state transition occurs only after actual compilation and backtest evidence exists.

Before installation, the user should preserve the generated files and their hashes as one experiment package. In QuantConnect Cloud, the Python project should contain the algorithm module and configuration module at the expected project root. Any copied supporting artifact should retain its original name and content. Manual edits after generation create a new implementation state and should be either prohibited or documented with a new hash and review.

The first platform objective is compilation, not performance. Import errors, unsupported APIs, missing data, or version differences should be resolved without changing strategy semantics. If a fix alters logic, the compiler or source contract should be updated and the project regenerated rather than patching only the cloud copy. This prevents the reviewed local artifact from diverging from the tested artifact.

A short diagnostic backtest can then exercise initialization, warm-up, one or more training events, reconstruction, corporate-action handlers where available, risk records, order events, and Object Store persistence. Diagnostic logs should be detailed enough to reproduce intermediate decisions but bounded enough to remain usable. After the path is verified, the declared full backtest can be run under the same configuration.

Review should proceed layer by layer. Confirm the subscribed universe and benchmark; inspect data normalization and history sufficiency; check model sample sizes and validation dates; review candidate scores and champion history; examine selected names and preliminary weights; verify position, sector, gross, and drawdown controls; and reconcile targets with orders and fills. Standard platform statistics are the final summary, not the first diagnostic.

Every rerun should have a reason. Examples include correcting a semantic defect, testing a prespecified sensitivity, or evaluating a new approved specification. Results, including failures and unattractive outcomes, belong in the experiment registry. The registry should distinguish compilation success, backtest completion, reconciliation status, falsification findings, and promotion review. A Boolean “success” field is too coarse for a governed research process.

If the strategy advances, the workflow expands rather than skips controls. Out-of-sample periods, paper trading, broker-model checks, borrow availability, operational monitoring, and independent review become necessary. The source currently refuses live mode, so enabling live behavior would require a deliberate new design, authority decision, and implementation review. Copying the research code into a live project would violate the stated boundary.

26 A More Institutional Roadmap

The present bridge can evolve substantially without changing its organizing principle. The static teaching universe can become a governed point-in-time universe. The model library can add gradient boosting, regularized regressions, calibrated classifiers, specialized ranking losses, or sector-specific models. The feature contract can expand to fundamentals, macro data, market microstructure, or textual signals. The portfolio engine can become covariance-aware and factor neutral. Risk can move into an independently permissioned service. Audit records can become cryptographically chained. Cost models can incorporate borrow and liquidity constraints. The validation engine can become a complete walk-forward research protocol.

The key requirement is that these expansions enter through explicit contracts and approved capabilities. More autonomy should not mean more hidden authority. In fact, as the number of models, data sources, agents, and experiments grows, the importance of machine-readable specifications, reproducible code generation, role separation, and falsification increases.

An institutional roadmap should grow evidence and controls alongside analytical capability. The first stage is research hardening: point-in-time universe data, reproducible environments, walk-forward validation, cost and borrow scenarios, factor attribution, and systematic reconciliation tests. These improvements address whether the existing hypothesis is measured honestly before expanding the search space.

The second stage is portfolio and risk hardening. Covariance-aware construction, beta and factor constraints, liquidity-aware sizing, turnover budgets, stress testing, and explicit restart rules can make exposures more intentional. Risk should move from a colocated logical component toward independently permissioned controls that cannot be modified by the forecasting process. Broker and portfolio state should be reconciled continuously rather than only at termination.

The third stage is operational hardening. Immutable artifact storage, signed builds, environment versioning, automated test gates, incident procedures, monitoring, secrets management, and controlled releases become necessary as the consequences of failure increase. Research, staging, paper, and live environments should have distinct permissions. Promotion should refer to a specific artifact hash and evidence package, not a mutable project name.

The fourth stage can expand autonomous research. Agents may coordinate data checks, experiment generation, test execution, evidence comparison, and report drafting, provided their tool access remains bounded and every consequential output passes deterministic validation. Budget limits and stopping rules should prevent uncontrolled experiment proliferation. Negative results and abandoned branches should remain discoverable so that the organization does not repeatedly relearn the same failure.

Institutional maturity is not measured by the number of models or agents. It is measured by the organization's ability to explain what was tested, reproduce how it was tested, identify who authorized each transition, detect when behavior diverges from policy, and stop safely. The architecture in this paper is useful because it establishes those questions early, while the implementation remains small enough to inspect. Later sophistication should preserve that clarity.

At every stage, the roadmap should include explicit exit criteria. An experiment that cannot meet data-quality, stability, cost, risk, or operational thresholds should be stopped or returned to research. Advancement is justified by accumulated evidence, not by the sunk cost of increasingly elaborate infrastructure. This discipline keeps institutionalization from becoming a process that merely automates persistence with a weak idea.

27 Conclusion

For a QuantConnect practitioner who does not wish to follow the entire notebook progression, the final implementation can be understood as a governed, adaptive, cross-sectional machine-learning strategy implemented natively in LEAN.

Thirty U.S. equities generate trailing momentum, volatility, trading-activity, and trend features. Five heterogeneous model families compete on historical evidence. A chronological validation rule selects the current champion. The champion generates cross-sectional scores. The strongest securities become long candidates and the weakest become short candidates. Inverse volatility converts those selections into preliminary exposures. Independent risk constrains positions, sector concentration, gross exposure, and portfolio drawdown. LEAN supplies sequential market data, schedules, corporate-action events, order mechanics, fills, portfolio accounting, slippage, and persistent audit storage.

The deeper contribution is architectural. The system is explicitly designed around the possibility that every model can be wrong. That possibility motivates model competition. Competition motivates validation. Validation motivates provenance. Portfolio implementation motivates cost realism. Capital allocation motivates independent risk. LLM-assisted research motivates capability boundaries. Event-driven deployment motivates deterministic compilation. And execution capability motivates explicit authority controls.

The resulting proposition is simple but consequential:

AI does not replace the quantitative investment institution. It expands the institution's ability to search, compare, learn, and criticize. Deterministic specifications, mathematical selection, portfolio construction, independent risk, LEAN execution mechanics, audit evidence, and human authority determine what the institution is actually allowed to do.

NB07 is therefore not merely a port of a machine-learning strategy to QuantConnect. It is the implementation bridge between an autonomous quantitative research architecture and a governed event-driven trading system.

The immediate deliverable is therefore a governed research artifact, not a claim of trading readiness. Its value lies in making the complete decision path inspectable: a bounded mission becomes a validated specification; the specification becomes deterministic LEAN code; approved models compete on chronological evidence; the champion supplies relative scores; portfolio construction assigns preliminary capital; independent risk can veto that capital; and platform events create execution evidence for review. Every arrow in this sequence carries both information and responsibility.

The design also clarifies how to interpret failure. If no model beats a simple baseline, the research hypothesis is weak. If features disagree across environments, semantic translation is defective. If costs erase returns, the portfolio is economically fragile. If approved targets and fills diverge, execution assumptions require investigation. If audit evidence is incomplete, the run cannot support promotion. These outcomes are useful because they point to different remedies and discourage indiscriminate parameter adjustment.

Success should be expressed with equal precision. Compilation proves syntactic platform compatibility. A completed backtest proves that the event path ran under a stated environment. Reconciliation supports semantic fidelity. Falsification tests provide bounded evidence of robustness. Paper or staged operation provides operational evidence. None alone grants the next level of authority. The registry and human review process preserve these distinctions.

The strategy's model tournament is ultimately one replaceable analytical component. Different features, estimators, or portfolio methods may later prove superior. The durable contribution is the architecture that lets those alternatives be proposed, constrained, compared, executed, and criticized without allowing any one component to define its own

permissions. That structure is especially important when generative AI increases the speed at which hypotheses and code can be produced.

For the QuantConnect practitioner, the practical message is straightforward: inspect the contract before the curve, inspect intermediate decisions before aggregate statistics, and preserve evidence before drawing conclusions. LEAN supplies a realistic event-driven environment, but governance must specify what the algorithm is allowed to learn, hold, risk, and record. The bridge joins those responsibilities into one reviewable experiment while intentionally leaving live authority outside its scope.

References

- [1] QuantConnect. *LEAN Documentation: Writing Algorithms*. 2026. URL: <https://www.quantconnect.com/docs/v2/writing-algorithms> (visited on 09/07/2026).
- [2] Thomas M. Cover and Peter E. Hart. “Nearest Neighbor Pattern Classification”. In: *IEEE Transactions on Information Theory* 13.1 (1967), pp. 21–27. DOI: [10.1109/TIT.1967.1053964](https://doi.org/10.1109/TIT.1967.1053964).
- [3] Leo Breiman. “Random Forests”. In: *Machine Learning* 45.1 (2001), pp. 5–32. DOI: [10.1023/A:1010933404324](https://doi.org/10.1023/A:1010933404324).
- [4] Harry Markowitz. “Portfolio Selection”. In: *The Journal of Finance* 7.1 (1952), pp. 77–91. DOI: [10.1111/j.1540-6261.1952.tb01525.x](https://doi.org/10.1111/j.1540-6261.1952.tb01525.x).
- [5] David H. Bailey et al. “The Probability of Backtest Overfitting”. In: *The Journal of Computational Finance* 20.4 (2017), pp. 39–69. DOI: [10.21314/JCF.2016.322](https://doi.org/10.21314/JCF.2016.322).

A Algorithmic Summary

1. Admit a research-only mission and reject live/deployment authority.
2. Validate the proposed model, feature, and portfolio tools against allowlists.
3. Compile the proposal into a deterministic `StrategySpec` and configuration hash.
4. Initialize LEAN with the declared universe, adjusted daily data, benchmark, reality models, warm-up, and scheduled callbacks.
5. On each monthly training date, request 320 trailing daily bars, construct the feature panel, create one-day-ahead labels, and split the pooled chronological rows 80/20.
6. Fit the five approved models and calculate validation balanced accuracy for each candidate.
7. Select the champion deterministically using highest score and the fixed tie-break order.
8. At the monthly rebalance, compute current features, remove securities under recent corporate-action cool-down, and score the remaining cross section with the champion.
9. Select approximately the top 20% as longs and bottom 20% as shorts.
10. Construct inverse-volatility weights with approximately +0.5 and -0.5 gross exposure on the two sides.
11. Send the proposal to the independent risk governor for position, sector, gross, and drawdown review.
12. Call `set_holdings` using only approved weights, assigning zero to symbols absent from the approved portfolio.
13. Record corporate actions, champion selection, rebalances, and order events.
14. At algorithm termination, save an audit bundle to Object Store.
15. Reconcile platform evidence against the strategy contract; retain human promotion authority.

The summary is intentionally procedural. It states the expected order of decisions and the safe outcome at each authority boundary. Implementers can use it as a control-flow checklist, while reviewers can use it to select audit records for inspection. The steps should not be collapsed merely because several occur inside one callback. Training, prediction, portfolio construction, risk approval, and execution remain distinct decisions even when they execute within milliseconds of one another.

Inputs and outputs can be associated with every step. Mission admission outputs an authority state. Compilation outputs a validated specification and source artifacts. Initialization outputs declared subscriptions and schedules. Training outputs fitted candidate state and diagnostics. Selection outputs a champion identifier. Ranking outputs scores and tail membership. Portfolio construction outputs proposed weights. Risk outputs approved weights and reasons. LEAN outputs orders, fills, holdings, and accounting. Termination outputs the audit bundle. This chain provides a compact data lineage for the entire experiment.

Failure behavior is part of the summary as well. Invalid capabilities stop compilation. Insufficient history prevents training or scoring. A failed candidate is recorded rather than replaced secretly. An absent champion prevents reconstruction. An undersized eligible universe prevents malformed long and short groups. Binding constraints scale or eliminate exposures. Live mode raises an error. Persistence failure is reported. Specifying these behaviors prevents operational exceptions from becoming unreviewed strategy choices.

The sequence also highlights which activities occur once, periodically, or continuously. Mission validation and compilation occur before the platform run. Initialization occurs once. Training and reconstruction occur monthly under the declared schedule. Data, corporate-action, risk-state, and order events occur as the engine advances. Audit records accumulate throughout and are persisted at termination. Different cadences are coordinated through explicit state rather than assumed simultaneity.

When extending the algorithm, the summary should be revised before or alongside the code. A new feature source adds data and timing steps. A new optimizer adds constraints and diagnostics. A new risk rule adds an independent decision and evidence requirement. A new deployment stage adds authority and operational controls. If an implementation change cannot be represented clearly in this sequence, its ownership and verification may not yet be well defined.

B Compiled QuantConnect Configuration

The configuration module below is the compact executable representation of the governed experiment. It centralizes values that would otherwise be scattered across callback methods and model constructors. A reviewer should read it as a contract surface: changing a value can alter the population studied, the information set, the candidate tournament, the portfolio, or the loss controls. The module should therefore be versioned and hashed together with the generated algorithm.

Universe constants identify the thirty tradable equities and their sector assignments. Their ordering is kept stable for reproducible iteration and serialization, even though economic ranking is determined later by model scores. The benchmark is declared separately so that performance comparison does not silently add a tradable asset. Data normalization and resolution choices determine the historical observations supplied to features, and thus belong beside the universe rather than being treated as harmless platform defaults.

Feature configuration declares the canonical names and lookbacks used during both training and inference. These entries must agree with the code that constructs momentum, volatility, activity, and trend variables. The longest window also helps determine the required warm-up and history request. If a new feature is added to this list without an implementation, compilation or testing should fail; if code computes an undeclared feature, the mapping review should flag it as ungoverned behavior.

Model configuration names the approved candidate families, their fixed hyperparameters, the random seed, and the deterministic tie-break priority. These values define the scope of runtime adaptation. The tournament may select among listed candidates, but it may not discover or import an arbitrary estimator. Hyperparameters are fixed experiment inputs rather than results of hidden optimization. Any later tuning campaign should produce a new configuration and evidence package.

Training and validation entries define the history length, schedule, split proportion, target horizon, and comparison metric. Their interaction determines which rows can influence a champion decision. Reviewers should confirm that enough data remains after all rolling windows and label shifts, and that the later validation observations are never used to fit preprocessing. A change in these fields can alter both sample size and the interpretation of “best model.”

Signal and portfolio entries specify the tail fraction, volatility weighting, and side-level gross objectives. They convert model outputs into a bounded proposal without allowing raw score magnitude to set leverage. Risk entries then specify maximum absolute position, sector gross, total gross, and drawdown halt. These are independent constraints: satisfying one does not imply satisfying the others. The runtime governor applies them deterministically and reports any modification.

Execution and governance entries complete the contract. Fees and volume-share slippage expose the assumptions used for simulated fills. Corporate-action cool-down declares when a security becomes temporarily ineligible. Audit identifiers and persistence settings connect runtime evidence to the configuration. Authority fields prohibit live trading and preserve human promotion review. A configuration that omitted these denials would not be equivalent, even if its statistical logic were unchanged.

The listing is included verbatim so that readers can compare prose claims with actual constants. In use, automated validation should check types, ranges, uniqueness, registry membership, and cross-field consistency before LEAN initialization. Human reviewers should focus on economic rationale and whether the declared scope matches the intended experiment. Together, machine validation and review make the configuration a controlled interface rather than a convenient collection of parameters.

```

1 # Auto-generated by NB07. Research/backtest only.
2
3 SEED = 20260906
4 RESEARCH_ONLY = True
5 DEPLOYMENT_AUTHORIZED = False
6 LIVE_TRADING_AUTHORIZED = False
7

```

```

8 START_DATE = (2021, 1, 1)
9 END_DATE = (2025, 12, 31)
10 STARTING_CASH = 1_000_000
11 BENCHMARK = "SPY"
12
13 RESOLUTION = "DAILY"
14 DATA_NORMALIZATION = "ADJUSTED"
15 WARMUP_BARS = 80
16 TRAINING_LOOKBACK_BARS = 320
17 RETRAIN_FREQUENCY = "MONTHLY"
18 REBALANCE_FREQUENCY = "MONTHLY"
19
20 FEATURES = [
21     "ret_1",
22     "mom_5",
23     "mom_20",
24     "vol_20",
25     "vol_60",
26     "volume_z20",
27     "dollar_volume_z20",
28     "ma_ratio_20",
29     "ma_ratio_50",
30 ]
31
32 MODEL_FAMILIES = [
33     "logistic_regression",
34     "knn",
35     "random_forest",
36     "mlp",
37     "linear_regression",
38 ]
39 CHAMPION_SELECTION_RULE = (
40     "highest chronological validation balanced accuracy; "
41     "tie-break by predefined model-complexity order"
42 )
43
44 SIGNAL_MODE = "CROSS_SECTIONAL_LONG_SHORT"
45 LONG_FRACTION = 0.20
46 SHORT_FRACTION = 0.20
47 MIN_SIGNAL_ABS = 0.00
48
49 PORTFOLIO_METHOD = "inverse_volatility"
50 TARGET_DAILY_VOL = 0.010
51 MAX_GROSS = 1.00
52 MAX_POSITION_WEIGHT = 0.10
53 MAX_SECTOR_GROSS = 0.35
54 MAX_PORTFOLIO_DRAWDOWN = 0.12
55 EVENT_COOLDOWN_DAYS = 2
56
57 PER_ORDER_FEE_USD = 0.00
58 VOLUME_SHARE_LIMIT = 0.025
59 VOLUME_SHARE_PRICE_IMPACT = 0.10
60
61 UNIVERSE_BY_SECTOR = {
62     "Technology": ["AAPL", "MSFT", "NVDA", "AVGO", "ORCL"],
63     "Financials": ["JPM", "BAC", "GS", "MS", "C"],
64     "Healthcare": ["JNJ", "UNH", "LLY", "MRK", "ABBV"],
65     "Industrials": ["CAT", "GE", "HON", "UPS", "RTX"],
66     "Consumer": ["AMZN", "WMT", "COST", "MCD", "HD"],
67     "Energy": ["XOM", "CVX", "COP", "SLB", "EOG"],
68 }
69
70 AUDIT_OBJECT_STORE_KEY = "NB07/audit_bundle.json"

```

Listing 1: Notebook-derived strategy_config.py.

C Generated LEAN Algorithm

The generated algorithm below is the operational realization of the specification. Its structure should be evaluated by responsibility rather than merely by line count. Initialization establishes the platform environment and rejects unauthorized live use. Helper methods retrieve history and build point-in-time features. Model methods construct and fit only approved candidates. Scheduled callbacks manage training and portfolio reconstruction. Event handlers update corporate-action state and capture order evidence. Termination persists the audit bundle.

Initialization is a consequential phase because it fixes the simulation context. The algorithm sets dates and capital, adds benchmark and universe securities, applies adjusted daily data, attaches fee and slippage models, initializes the risk governor and audit recorder, and registers schedules. A failure here should stop the run visibly. Defaults inherited accidentally from the platform can change economics, so every material setting should be explicit or documented as intentionally delegated to LEAN.

History and feature methods form the temporal core. They request observations relative to current simulated time, validate sufficient length, calculate the nine declared features in canonical order, and exclude invalid values. Training-panel construction additionally creates the one-day-ahead target and removes rows whose outcome is unavailable. Current scoring uses the latest admissible completed observations without attaching a future label. The distinction between these two uses is central to preventing leakage.

The model laboratory builds each estimator with declared preprocessing and hyperparameters. During training, it fits on the earlier segment and evaluates on the later chronological segment. Candidate diagnostics are retained even when a model fails. Balanced accuracy and the fixed tie-break rule yield one champion, whose fitted pipeline is stored with its identity and training timestamp. The source should never choose a candidate through incidental container ordering.

Portfolio reconstruction begins only when champion and data state are valid. It checks cool-down and tradability, computes feature vectors, produces comparable scores, ranks the eligible cross section, selects disjoint tails, and estimates volatility. The constructor normalizes reciprocal-volatility weights to the side targets. It then sends the complete proposed map to the governor rather than calling the order interface directly.

The governor clamps positions, aggregates and limits sector gross, constrains total gross, and can return zero exposure after a drawdown breach. The execution method translates only approved targets into holdings instructions, including zeros for stale positions. LEAN remains responsible for quantities, order processing, fills, cash, and holdings. Order-event records connect the desired portfolio with what the simulator actually achieved.

Dividend and split handlers do not recalculate historical economics manually. They update eligibility cool-down state and write structured evidence while adjusted data continues to govern feature history. This separation avoids double adjustment and makes event-related exclusions visible. Termination assembles configuration identity, training and champion records, risk decisions, corporate actions, and order events for Object Store persistence.

The listing is long because it makes the bridge inspectable in one document. Reviewers need not trust the architectural description alone; they can locate the exact implementation of each mapped responsibility. When modifying the source, the safest workflow is to change the governed specification or compiler, regenerate the file, rerun validation, and compare hashes. Direct edits may be appropriate during debugging, but they create a distinct artifact that must be documented and reconciled before its results are treated as evidence.

```

1 from AlgorithmImports import *
2 from strategy_config import *
3
4 import json
5 import hashlib
6 from datetime import datetime
7
8 import numpy as np
9 import pandas as pd
10
```

```

11 from sklearn.pipeline import Pipeline
12 from sklearn.preprocessing import StandardScaler
13 from sklearn.linear_model import LogisticRegression, LinearRegression
14 from sklearn.neighbors import KNeighborsClassifier
15 from sklearn.ensemble import RandomForestClassifier
16 from sklearn.neural_network import MLPClassifier
17 from sklearn.metrics import balanced_accuracy_score
18
19
20 def stable_hash(value):
21     payload = json.dumps(value, sort_keys=True, default=str, separators=(",", ":"))
22     return hashlib.sha256(payload.encode("utf-8")).hexdigest()
23
24
25 class IndependentRiskGovernor:
26     """
27     Logically independent risk review.
28     It does not fit models and does not choose the research champion.
29     It receives candidate weights and can clamp or halt them.
30     """
31
32     def __init__(self):
33         self.peak_value = None
34         self.halted = False
35         self.last_report = {}
36
37     def review(self, algorithm, candidate_weights):
38         value = float(algorithm.portfolio.total_portfolio_value)
39
40         if self.peak_value is None:
41             self.peak_value = value
42         self.peak_value = max(self.peak_value, value)
43
44         drawdown = 0.0 if self.peak_value <= 0 else value / self.peak_value - 1.0
45         if drawdown <= -MAX_PORTFOLIO_DRAWDOWN:
46             self.halted = True
47
48         if self.halted:
49             self.last_report = {
50                 "status": "HALT",
51                 "portfolio_value": value,
52                 "drawdown": drawdown,
53                 "reason": "portfolio_drawdown_limit",
54             }
55             return {}
56
57         bounded = {}
58         for symbol, weight in candidate_weights.items():
59             bounded[symbol] = float(
60                 max(-MAX_POSITION_WEIGHT, min(MAX_POSITION_WEIGHT, weight))
61             )
62
63         # Sector concentration control.
64         for sector, symbols in algorithm._symbols_by_sector.items():
65             sector_symbols = [s for s in symbols if s in bounded]
66             sector_gross = sum(abs(bounded[s]) for s in sector_symbols)
67             if sector_gross > MAX_SECTOR_GROSS and sector_gross > 0:
68                 scale = MAX_SECTOR_GROSS / sector_gross
69                 for s in sector_symbols:
70                     bounded[s] *= scale
71
72         gross = sum(abs(w) for w in bounded.values())
73         if gross > MAX_GROSS and gross > 0:
74             scale = MAX_GROSS / gross
75             bounded = {s: w * scale for s, w in bounded.items()}
76
77         self.last_report = {
78             "status": "PASS",
79             "portfolio_value": value,
80             "drawdown": drawdown,
81             "gross": sum(abs(w) for w in bounded.values()),

```

```

82         "net": sum(bounded.values()),
83         "positions": len([w for w in bounded.values() if abs(w) > 1e-12]),
84     }
85     return bounded
86
87
88 class GovernedQuantConnectBridge(QCAAlgorithm):
89     """
90     NB07 QuantConnect/LEAN bridge.
91
92     The algorithm is intentionally research-only. The LLM/control plane is external
93     to this runtime. LEAN receives only deterministic configuration and code.
94     """
95
96     def initialize(self):
97         if self.live_mode or LIVE_TRADING_AUTHORIZED or DEPLOYMENT_AUTHORIZED:
98             raise RuntimeError(
99                 "NB07 governance denial: live trading/deployment is not authorized."
100             )
101
102         self.set_start_date(*START_DATE)
103         self.set_end_date(*END_DATE)
104         self.set_cash(STARTING_CASH)
105
106         self._audit = []
107         self._event_dates = {}
108         self._model_diagnostics = []
109         self._champion_name = None
110         self._champion_model = None
111         self._champion_kind = None
112         self._risk = IndependentRiskGovernor()
113
114         self._benchmark_symbol = self.add_equity(
115             BENCHMARK,
116             Resolution.DAILY,
117             data_normalization_mode=DataNormalizationMode.ADJUSTED
118         ).symbol
119         self.set_benchmark(self._benchmark_symbol)
120
121         self._symbols = []
122         self._sector_by_symbol = {}
123         self._symbols_by_sector = {}
124
125         for sector, tickers in UNIVERSE_BY_SECTOR.items():
126             sector_symbols = []
127             for ticker in tickers:
128                 security = self.add_equity(
129                     ticker,
130                     Resolution.DAILY,
131                     data_normalization_mode=DataNormalizationMode.ADJUSTED
132                 )
133                 security.set_fee_model(ConstantFeeModel(PER_ORDER_FEE_USD))
134                 security.set_slippage_model(
135                     VolumeShareSlippageModel(
136                         VOLUME_SHARE_LIMIT,
137                         VOLUME_SHARE_PRICE_IMPACT
138                     )
139                 )
140                 symbol = security.symbol
141                 self._symbols.append(symbol)
142                 sector_symbols.append(symbol)
143                 self._sector_by_symbol[symbol] = sector
144                 self._event_dates[symbol] = {
145                     "dividend": None,
146                     "split": None,
147                 }
148             self._symbols_by_sector[sector] = sector_symbols
149
150         self.set_warm_up(WARMUP_BARS, Resolution.DAILY)
151
152         # Model training runs inside LEAN's training mechanism.

```

```

153     self.train(
154         self.date_rules.month_start(self._benchmark_symbol),
155         self.time_rules.before_market_open(self._benchmark_symbol, 60),
156         self._train_models,
157     )
158
159     # Portfolio rebalance follows training on the same monthly schedule.
160     self.schedule.on(
161         self.date_rules.month_start(self._benchmark_symbol),
162         self.time_rules.after_market_open(self._benchmark_symbol, 30),
163         self._rebalance,
164     )
165
166     self._record(
167         "INITIALIZED",
168         {
169             "universe_size": len(self._symbols),
170             "model_families": MODEL_FAMILIES,
171             "portfolio_method": PORTFOLIO_METHOD,
172             "research_only": RESEARCH_ONLY,
173         },
174     )
175
176     def _record(self, event_type, payload):
177         record = {
178             "time": str(self.time),
179             "event_type": event_type,
180             "payload": payload,
181         }
182         record["hash"] = stable_hash(record)
183         self._audit.append(record)
184
185     def _build_model(self, name):
186         if name == "logistic_regression":
187             return (
188                 Pipeline([
189                     ("scale", StandardScaler()),
190                     ("model", LogisticRegression(
191                         max_iter=500,
192                         random_state=SEED,
193                     )),
194                 ]),
195                 "classification",
196             )
197         if name == "knn":
198             return (
199                 Pipeline([
200                     ("scale", StandardScaler()),
201                     ("model", KNeighborsClassifier(
202                         n_neighbors=9,
203                         weights="distance",
204                     )),
205                 ]),
206                 "classification",
207             )
208         if name == "random_forest":
209             return (
210                 RandomForestClassifier(
211                     n_estimators=160,
212                     max_depth=6,
213                     min_samples_leaf=12,
214                     random_state=SEED,
215                     n_jobs=1,
216                 ),
217                 "classification",
218             )
219         if name == "mlp":
220             return (
221                 Pipeline([
222                     ("scale", StandardScaler()),
223                     ("model", MLPClassifier(

```

```

224         hidden_layer_sizes=(24, 12),
225         max_iter=180,
226         early_stopping=True,
227         random_state=SEED,
228     )),
229 ],
230     "classification",
231 )
232 if name == "linear_regression":
233     return (
234         Pipeline([
235             ("scale", StandardScaler()),
236             ("model", LinearRegression()),
237         ]),
238         "regression",
239     )
240     raise ValueError(f"Unknown approved model family: {name}")
241
242 def _feature_frame_for_symbol(self, frame):
243     frame = frame.sort_index().copy()
244     close = frame["close"].astype(float)
245     volume = frame["volume"].astype(float).replace(0, np.nan)
246     dollar_volume = (close * volume).replace(0, np.nan)
247
248     out = pd.DataFrame(index=frame.index)
249     out["ret_1"] = close.pct_change(1)
250     out["mom_5"] = close.pct_change(5)
251     out["mom_20"] = close.pct_change(20)
252     out["vol_20"] = close.pct_change().rolling(20).std()
253     out["vol_60"] = close.pct_change().rolling(60).std()
254
255     vol_mean = volume.rolling(20).mean()
256     vol_std = volume.rolling(20).std().replace(0, np.nan)
257     out["volume_z20"] = (volume - vol_mean) / vol_std
258
259     dv_mean = dollar_volume.rolling(20).mean()
260     dv_std = dollar_volume.rolling(20).std().replace(0, np.nan)
261     out["dollar_volume_z20"] = (dollar_volume - dv_mean) / dv_std
262
263     out["ma_ratio_20"] = close / close.rolling(20).mean() - 1.0
264     out["ma_ratio_50"] = close / close.rolling(50).mean() - 1.0
265
266     out["label_return_1"] = close.pct_change().shift(-1)
267     out["label_up"] = (out["label_return_1"] > 0).astype(int)
268     return out
269
270 def _build_training_dataset(self):
271     history = self.history(
272         self._symbols,
273         TRAINING_LOOKBACK_BARS,
274         Resolution.DAILY,
275     )
276     if history is None or history.empty:
277         return pd.DataFrame()
278
279     parts = []
280     for symbol in self._symbols:
281         try:
282             frame = history.loc[symbol]
283         except Exception:
284             continue
285
286         if frame is None or len(frame) < 80:
287             continue
288
289         features = self._feature_frame_for_symbol(frame)
290         features["symbol"] = str(symbol)
291         features["event_time"] = features.index
292         features = features.dropna(
293             subset=FEATURES + ["label_return_1"]
294         )

```

```

295         parts.append(features)
296
297     if not parts:
298         return pd.DataFrame()
299
300     dataset = pd.concat(parts, axis=0)
301     return dataset.sort_values("event_time").reset_index(drop=True)
302
303 def _train_models(self):
304     if self.live_mode:
305         raise RuntimeError("NB07 governance denial: live execution is prohibited.")
306     if self.is_warming_up:
307         return
308
309     dataset = self._build_training_dataset()
310     if dataset.empty or len(dataset) < 500:
311         self._record("TRAIN_SKIPPED", {"rows": int(len(dataset))})
312         return
313
314     cut = int(len(dataset) * 0.80)
315     train = dataset.iloc[:cut].copy()
316     valid = dataset.iloc[cut:].copy()
317
318     X_train = train[FEATURES].values
319     X_valid = valid[FEATURES].values
320     y_train_up = train["label_up"].astype(int).values
321     y_valid_up = valid["label_up"].astype(int).values
322     y_train_return = train["label_return_1"].astype(float).values
323
324     diagnostics = []
325     candidates = []
326
327     for complexity, name in enumerate(MODEL_FAMILIES):
328         model, kind = self._build_model(name)
329         try:
330             if kind == "classification":
331                 model.fit(X_train, y_train_up)
332                 pred_up = model.predict(X_valid).astype(int)
333             else:
334                 model.fit(X_train, y_train_return)
335                 pred_return = model.predict(X_valid).astype(float)
336                 pred_up = (pred_return > 0).astype(int)
337
338             score = float(balanced_accuracy_score(y_valid_up, pred_up))
339             diagnostics.append({
340                 "model": name,
341                 "kind": kind,
342                 "validation_balanced_accuracy": score,
343                 "complexity_rank": complexity,
344             })
345             candidates.append(
346                 (score, -complexity, name, model, kind)
347             )
348         except Exception as exc:
349             diagnostics.append({
350                 "model": name,
351                 "status": "FAILED",
352                 "error": str(exc),
353                 "complexity_rank": complexity,
354             })
355
356     if not candidates:
357         self._record("TRAIN_FAILED", {"diagnostics": diagnostics})
358         return
359
360     # Deterministic champion selection.
361     candidates.sort(key=lambda x: (x[0], x[1]), reverse=True)
362     score, _, name, model, kind = candidates[0]
363
364     self._champion_name = name
365     self._champion_model = model

```

```

366     self._champion_kind = kind
367     self._model_diagnostics = diagnostics
368
369     self._record(
370         "CHAMPION_SELECTED",
371         {
372             "champion": name,
373             "kind": kind,
374             "validation_balanced_accuracy": score,
375             "diagnostics": diagnostics,
376         },
377     )
378
379     def _latest_features(self):
380         history = self.history(
381             self._symbols,
382             max(80, WARMUP_BARS),
383             Resolution.DAILY,
384         )
385         if history is None or history.empty:
386             return pd.DataFrame()
387
388         rows = []
389         for symbol in self._symbols:
390             try:
391                 frame = history.loc[symbol]
392             except Exception:
393                 continue
394
395             if frame is None or len(frame) < 65:
396                 continue
397
398             features = self._feature_frame_for_symbol(frame)
399             latest = features.iloc[-1]
400             if latest[FEATURES].isna().any():
401                 continue
402
403             row = latest[FEATURES].to_dict()
404             row["symbol_object"] = symbol
405             rows.append(row)
406
407         return pd.DataFrame(rows)
408
409     def _recent_corporate_action(self, symbol):
410         dates = self._event_dates.get(symbol, {})
411         for event_date in dates.values():
412             if event_date is None:
413                 continue
414             if (self.time.date() - event_date).days <= EVENT_COOLDOWN_DAYS:
415                 return True
416         return False
417
418     def _model_scores(self, latest):
419         X = latest[FEATURES].values
420         if self._champion_kind == "classification":
421             if hasattr(self._champion_model, "predict_proba"):
422                 probability = self._champion_model.predict_proba(X)[: , 1]
423                 return probability - 0.5
424             return self._champion_model.predict(X).astype(float) - 0.5
425         return self._champion_model.predict(X).astype(float)
426
427     def _portfolio_weights(self, latest):
428         scores = self._model_scores(latest)
429         records = []
430
431         for i, row in latest.iterrows():
432             symbol = row["symbol_object"]
433             if self._recent_corporate_action(symbol):
434                 continue
435             records.append({
436                 "symbol": symbol,

```

```

437         "score": float(scores[i]),
438         "vol": max(float(row["vol_20"]), 1e-6),
439     })
440
441     if len(records) < 6:
442         return {}
443
444     records.sort(key=lambda x: x["score"])
445     n_long = max(1, int(len(records) * LONG_FRACTION))
446     n_short = max(1, int(len(records) * SHORT_FRACTION))
447
448     short_side = records[:n_short]
449     long_side = records[-n_long:]
450
451     weights = {}
452
453     if PORTFOLIO_METHOD == "equal_weight":
454         for r in long_side:
455             weights[r["symbol"]] = 0.5 / len(long_side)
456         for r in short_side:
457             weights[r["symbol"]] = -0.5 / len(short_side)
458
459     else:
460         # inverse-volatility / risk-parity proxy
461         long_inv = np.array([1.0 / r["vol"] for r in long_side], dtype=float)
462         short_inv = np.array([1.0 / r["vol"] for r in short_side], dtype=float)
463         long_inv = long_inv / long_inv.sum()
464         short_inv = short_inv / short_inv.sum()
465
466         for r, w in zip(long_side, long_inv):
467             weights[r["symbol"]] = float(0.5 * w)
468         for r, w in zip(short_side, short_inv):
469             weights[r["symbol"]] = float(-0.5 * w)
470
471         if PORTFOLIO_METHOD == "volatility_target":
472             approx_vol = np.sqrt(
473                 sum(
474                     (weights[r["symbol"]] * r["vol"]) ** 2
475                     for r in long_side + short_side
476                 )
477             )
478             if approx_vol > 0:
479                 scale = min(1.0, TARGET_DAILY_VOL / approx_vol)
480                 weights = {s: w * scale for s, w in weights.items()}
481
482     return weights
483
484 def _rebalance(self):
485     if self.live_mode:
486         raise RuntimeError("NB07 governance denial: live execution is prohibited.")
487     if self.is_warming_up or self._champion_model is None:
488         return
489
490     latest = self._latest_features()
491     if latest.empty:
492         return
493
494     proposed = self._portfolio_weights(latest)
495     approved = self._risk.review(self, proposed)
496
497     for symbol in self._symbols:
498         self.set_holdings(symbol, float(approved.get(symbol, 0.0)))
499
500     self._record(
501         "REBALANCE",
502         {
503             "champion": self._champion_name,
504             "proposed_positions": len(proposed),
505             "approved_positions": len(approved),
506             "risk_report": self._risk.last_report,
507         },

```

```

508     )
509
510     def on_dividends(self, dividends):
511         for symbol, dividend in dividends.items():
512             if symbol in self._event_dates:
513                 self._event_dates[symbol]["dividend"] = self.time.date()
514                 self._record(
515                     "DIVIDEND",
516                     {
517                         "symbol": str(symbol),
518                         "distribution": float(dividend.distribution),
519                     },
520                 )
521
522     def on_splits(self, splits):
523         for symbol, split in splits.items():
524             if symbol in self._event_dates:
525                 self._event_dates[symbol]["split"] = self.time.date()
526                 self._record(
527                     "SPLIT",
528                     {
529                         "symbol": str(symbol),
530                         "split_factor": float(split.split_factor),
531                         "type": str(split.type),
532                     },
533                 )
534
535     def on_order_event(self, order_event):
536         if order_event.status in [
537             OrderStatus.FILLED,
538             OrderStatus.PARTIALLY_FILLED,
539             OrderStatus.CANCELED,
540             OrderStatus.INVALID,
541         ]:
542             self._record(
543                 "ORDER_EVENT",
544                 {
545                     "order_id": order_event.order_id,
546                     "symbol": str(order_event.symbol),
547                     "status": str(order_event.status),
548                     "fill_quantity": float(order_event.fill_quantity),
549                     "fill_price": float(order_event.fill_price),
550                 },
551             )
552
553     def on_data(self, data):
554         # Explicitly keep live denial inside the event loop as a second boundary.
555         if self.live_mode:
556             raise RuntimeError("NB07 governance denial: live execution is prohibited.")
557
558     def on_end_of_algorithm(self):
559         bundle = {
560             "notebook": "NB07",
561             "bridge": "QuantConnect/LEAN",
562             "status": "BACKTEST_RESEARCH_ONLY",
563             "live_trading_authorized": False,
564             "deployment_authorized": False,
565             "champion": self._champion_name,
566             "model_diagnostics": self._model_diagnostics,
567             "risk_report": self._risk.last_report,
568             "audit_events": self._audit,
569             "audit_head": stable_hash(self._audit),
570             "configuration_hash": stable_hash({
571                 "models": MODEL_FAMILIES,
572                 "features": FEATURES,
573                 "portfolio_method": PORTFOLIO_METHOD,
574                 "universe": UNIVERSE_BY_SECTOR,
575             }),
576         }
577
578         try:

```

```
579         self.object_store.save(  
580             AUDIT_OBJECT_STORE_KEY,  
581             json.dumps(bundle, sort_keys=True, default=str),  
582         )  
583     except Exception as exc:  
584         self.log(f"NB07 Object Store warning: {exc}")  
585  
586     self.log(  
587         "NB07 COMPLETE: research-only QuantConnect backtest; "  
588         f"champion={self._champion_name}; live authority=DENIED"  
589     )
```

Listing 2: Notebook-derived main.py.